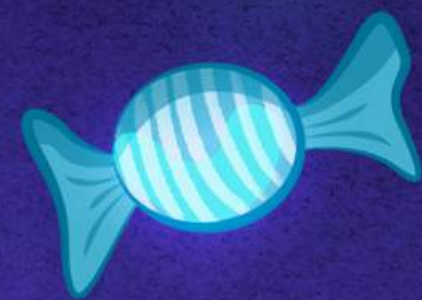
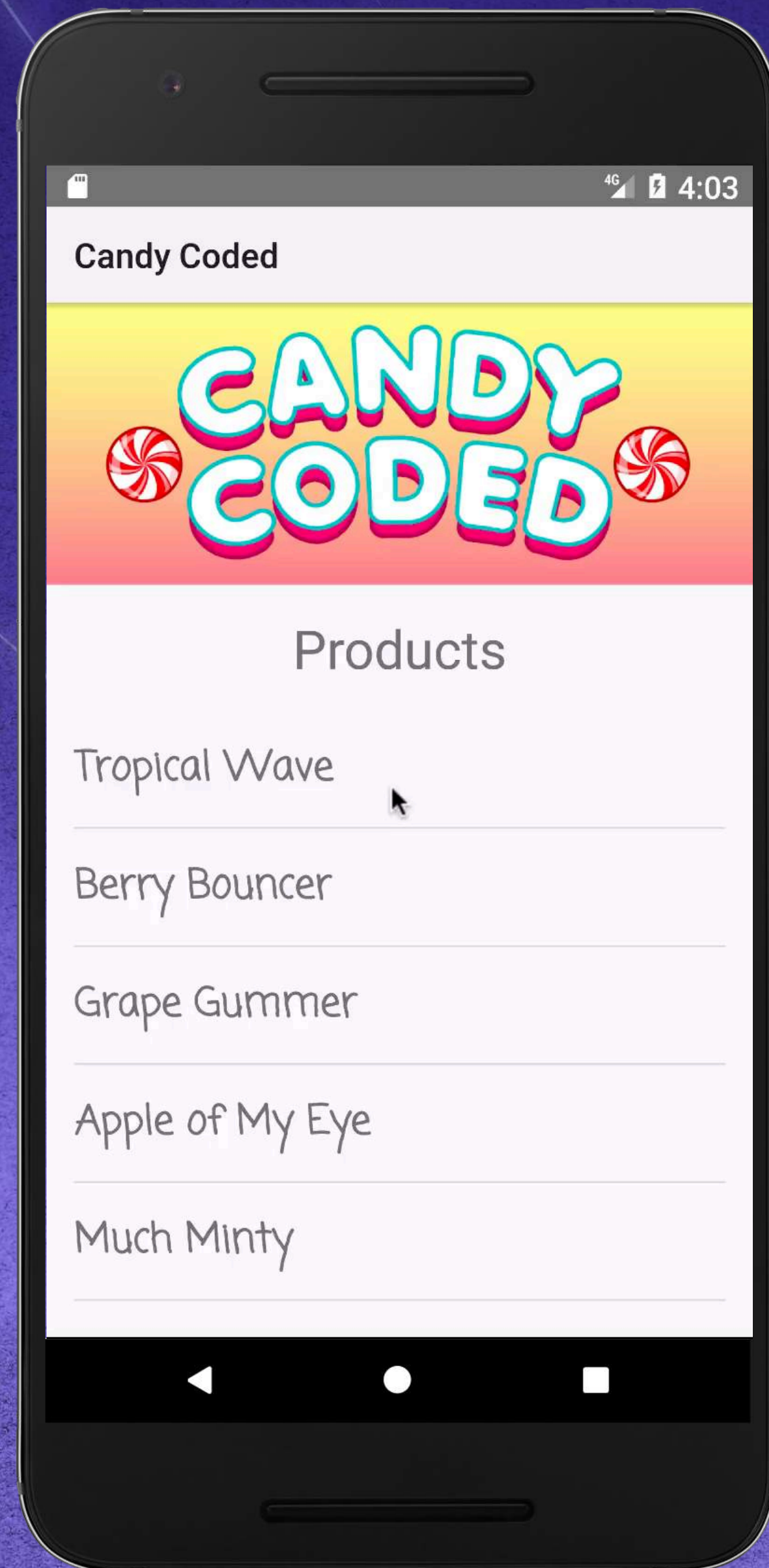


SUPER SWEET ANDROID TIME



Where We Left Off in Try Android



This is where the Candy Coded app left off in the Try Android course.



We created a candy store app and learned how to:

- Create an Activity **Layout**
- Add Images and an **ImageView**
- Add data to a **ListView with an Adapter**
- Create an **Event Listener**
- Display a **Toast**

SUPER
SWEET
ANDROID
TIME

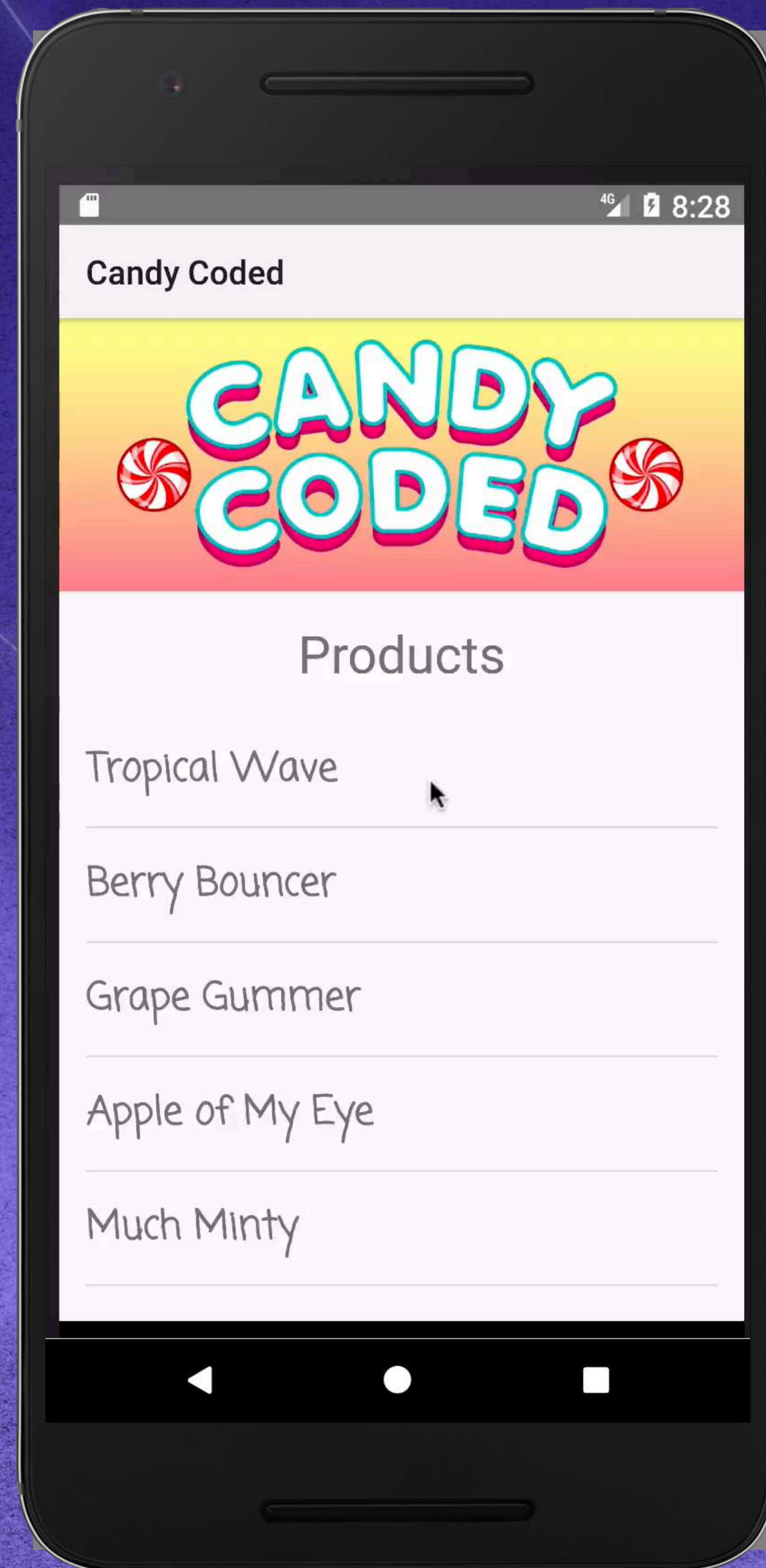


Screencast: Code Overview

SUPER
SWEET
ANDROID
TIME



The App We're Going to Build



In this course we're going to add more features to our Candy Coded app like:

- Launching another Activity **with an Intent**
- Performing a Network Request **with AsyncHttpClient**
- Parsing JSON into Objects **with GSON**
- **Designing a Constraint Layout**
- **Downloading and Cacheing Images with Picasso**
- **Storing Data in a Database**

You can download the finished Candy Coded app from the Try Android course here:
go.codeschool.com/TryAndroid-SampleApp



Level 1 Section 1

Activities and Intents

Launching an Activity with an Intent

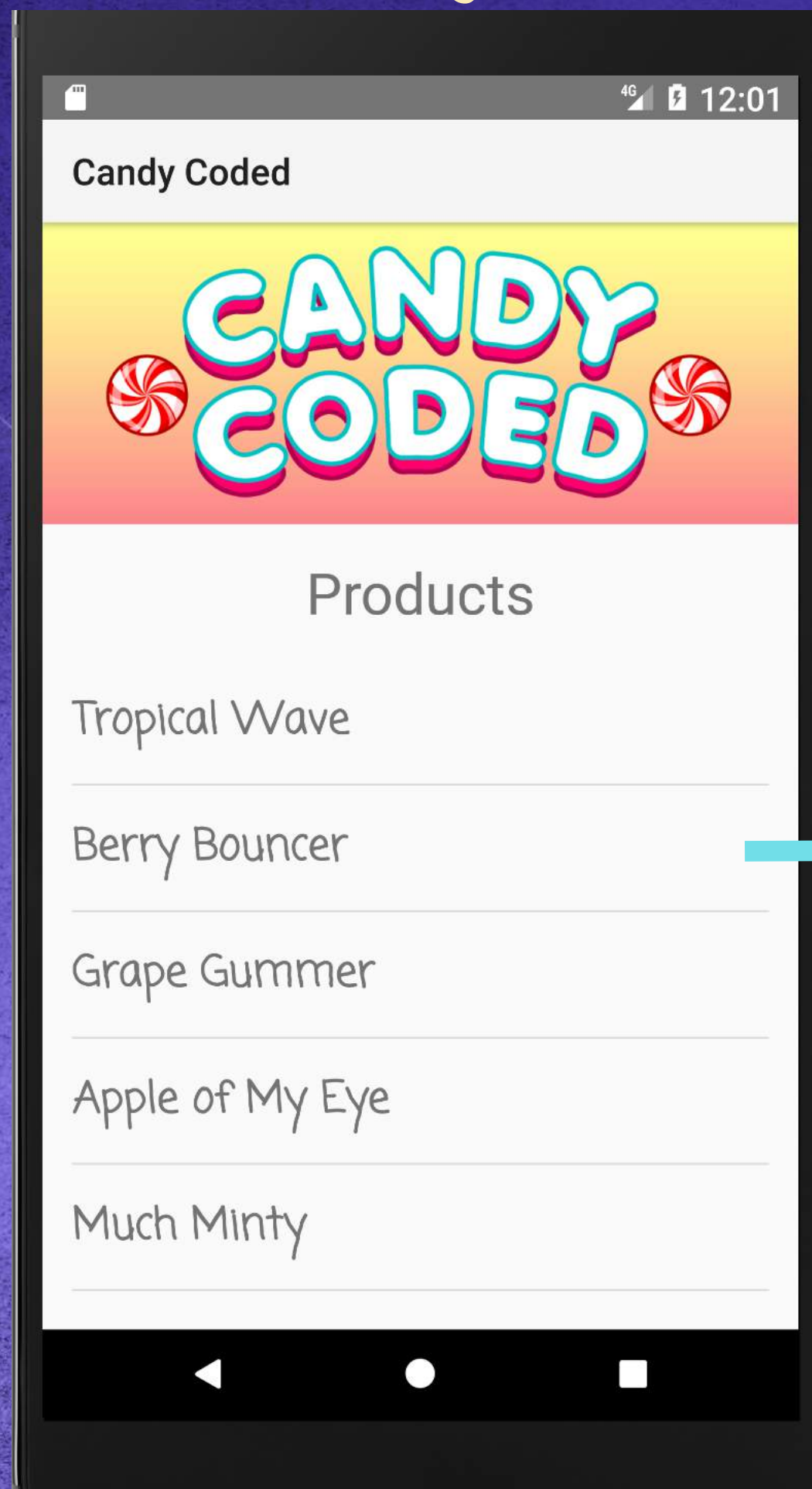
SUPER
SWEET
ANDROID
TIME



A New Activity will Show a Candy's Details

We want to create a new Activity called the DetailActivity to show the details of each candy.

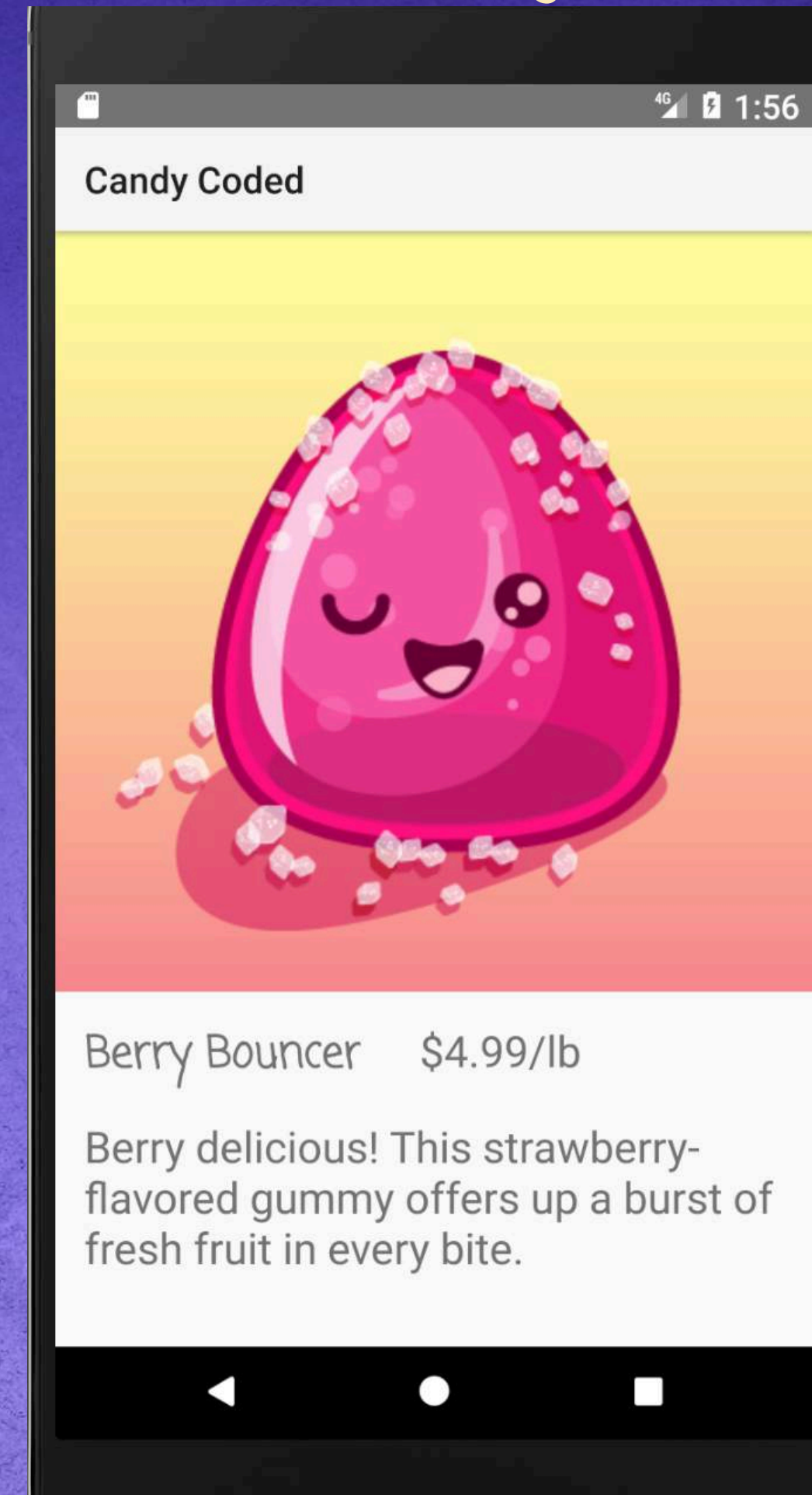
Main Activity



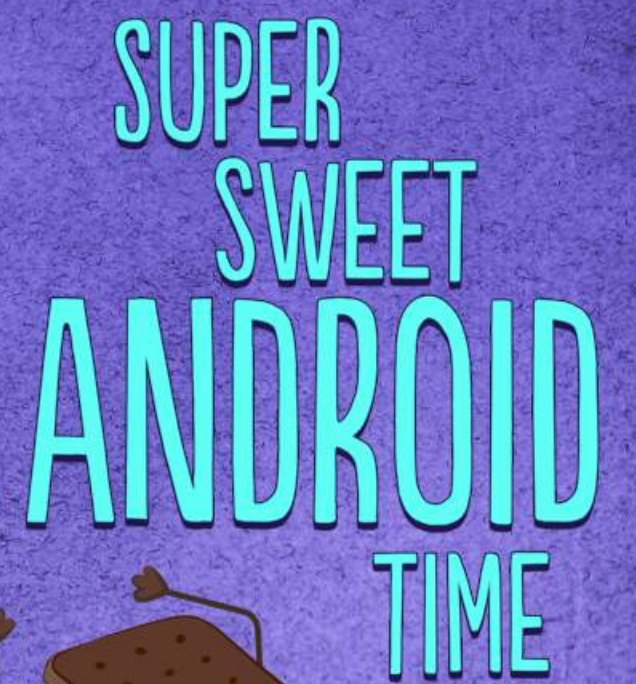
*When a list item
is clicked the
Detail Activity
will start*



Detail Activity



*To do this we need
to create a new
Activity to show.*



Screencast: How to Create a New Activity

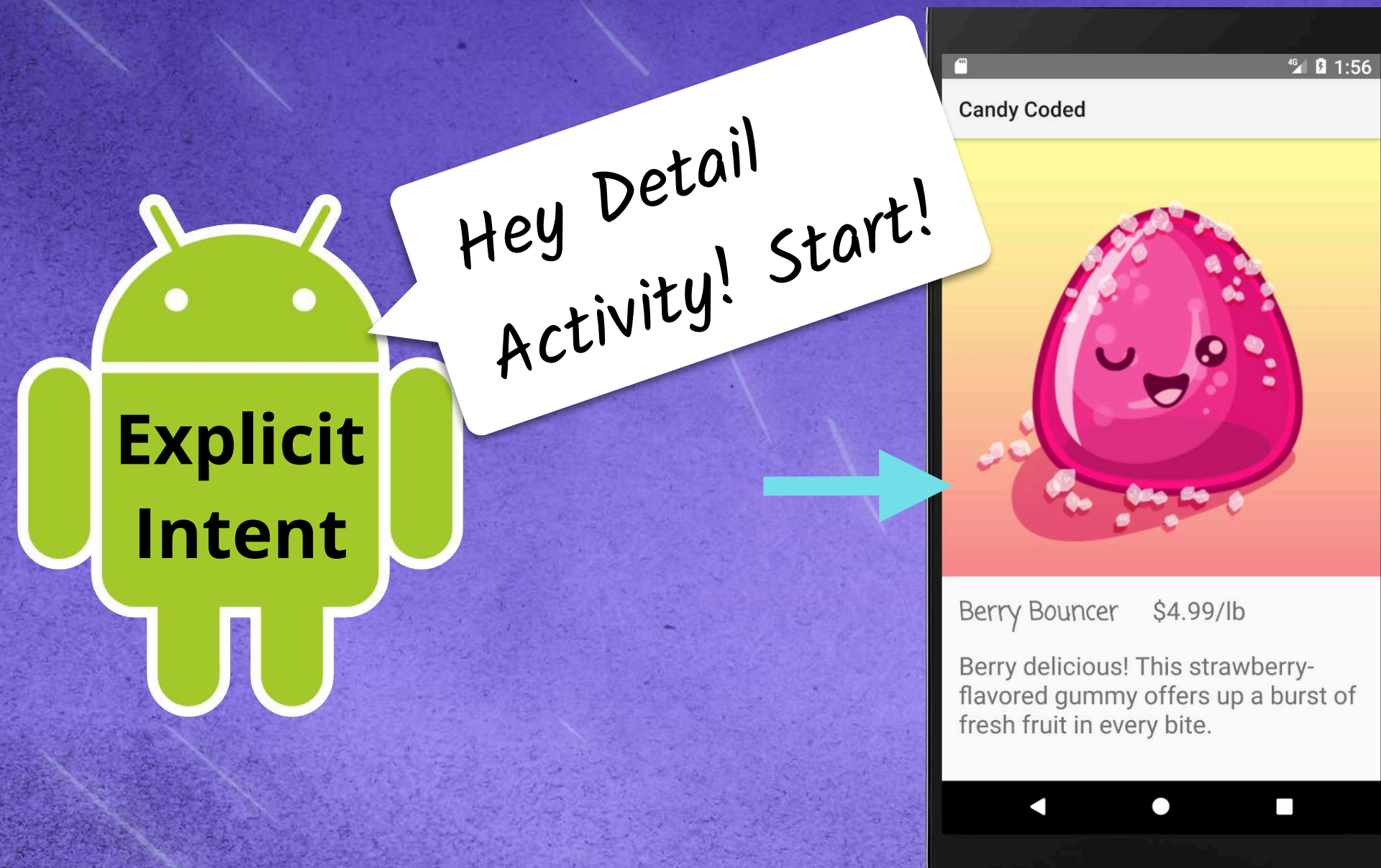
SUPER
SWEET
ANDROID
TIME



We Need an Intent to Launch Another Activity

An Intent requests an action from another app component.
There are two types of Intents - Explicit and Implicit.

Explicit intents launch specific app components like the Detail Activity in this case

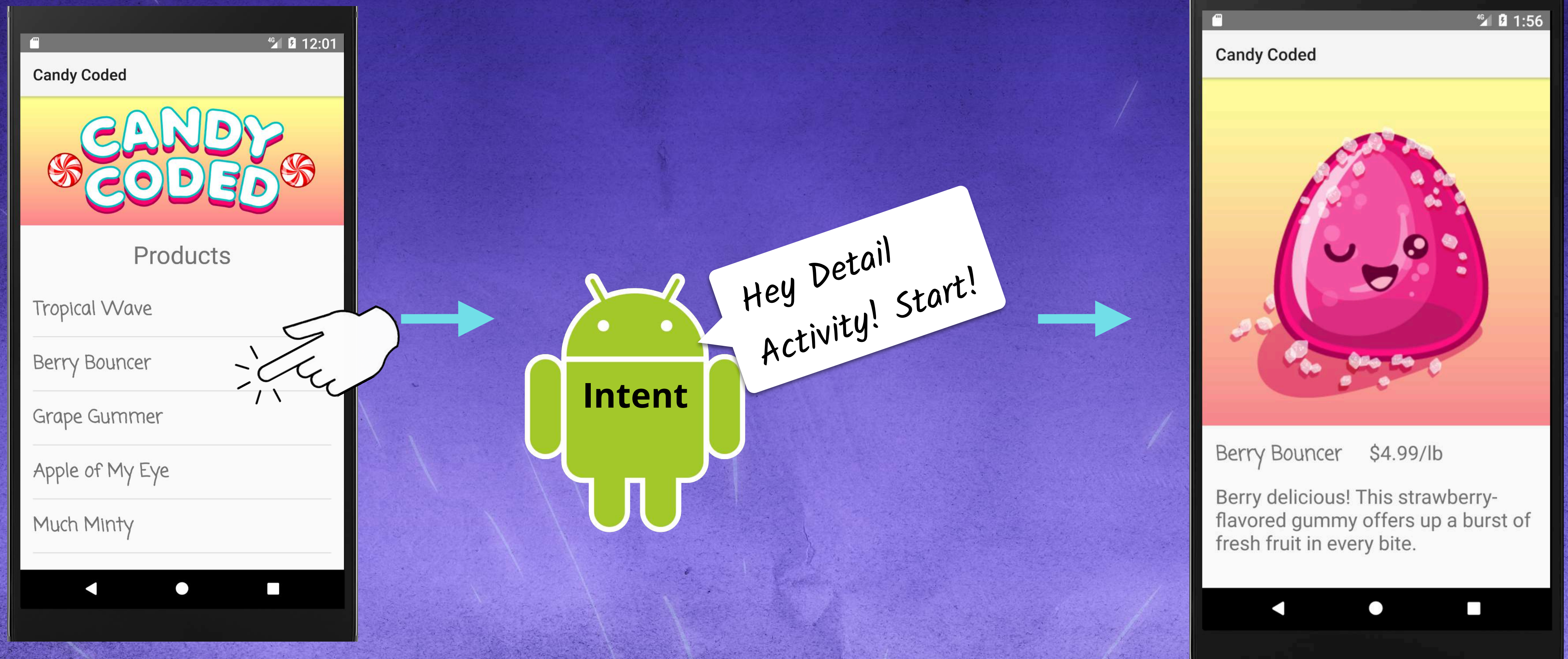


Implicit intents declare an action to perform, and the user can pick which app to use



We'll Use an Explicit Intent to Start the Detail Activity

When a User selects an item from our ListView our Item Click Listener will create an Intent to launch the Detail Activity.

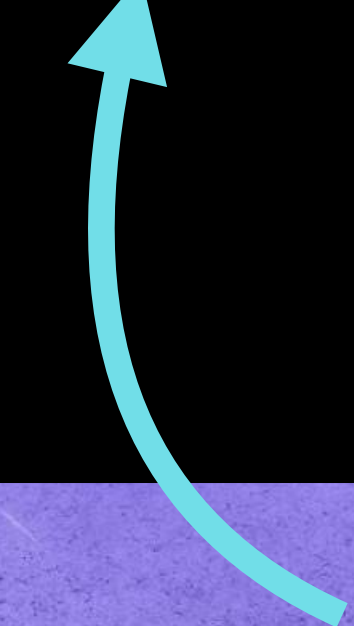


Creating the Intent in MainActivity.java

Since we want the Intent to start the Activity after a click we'll add that code to the Click Listener at the bottom of the onCreate() method in MainActivity.java.

MainActivity.java

```
...  
    listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {  
        @Override  
        public void onItemClick(  
            AdapterView<?> adapterView, View view, int i, long l) {  
              
        }  
    });  
...
```



Previously we were showing a Toast here, but we'll delete that code and only start our Detail Activity

Creating the Intent to Start the DetailActivity

To start an Activity we need an Intent created with the DetailActivity class we want to launch.

*The context of where we
are creating this Intent*

*The component used
in the Intent*

```
Intent detailIntent = new Intent(MainActivity.this,  
                                DetailActivity.class);  
startActivity(detailIntent);
```

*The MainActivity can start another Activity with the
startActivity() method, which takes an Intent as a parameter*

SUPER
SWEET
ANDROID
TIME



Creating the Intent in MainActivity.java

The Intent to start the Detail Activity **inside the** Click Listener **callback method** - onItemClick().

MainActivity.java

```
...
    listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(
            AdapterView<?> adapterView, View view, int i, long l) {
            Intent detailIntent = new Intent(MainActivity.this,
                                           DetailActivity.class);
            startActivity(detailIntent);
        }
    });
...
```


Screencast: Starting the DetailActivity

SUPER
SWEET
ANDROID
TIME



Level 1 Section 2

Activities and Intents

Passing Data with an Intent

SUPER
SWEET
ANDROID
TIME



How Can We Pass Data to an Activity?

We will need to pass data to our Detail Activity so that it knows what candy to show.



Passing Data to an Activity with an Intent

We can pass data in an Intent with the `putExtra()` method.

MainActivity.java

```
...  
Intent detailIntent = new Intent(MainActivity.this, DetailActivity.class);  
detailIntent.putExtra("candy_name", "Name of your candy here");  
startActivity(detailIntent);  
...
```

We need to assign a key to our data so we can look it up in the Activity

How do we know which candy item was selected from the list?

SUPER
SWEET
ANDROID
TIME



Passing the Selected Candy Name to the Detail Activity

Remember that our candy was stored in an `ArrayList`, so we can look up the candy to send to the Activity at position `i`.

MainActivity.java

```
...
final ArrayList<String> candyList = new ArrayList<String>();
candyList.add("Tropical Wave");
candyList.add("Berry Bouncer");
...
listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(
        AdapterView<?> adapterView, View view, int i, long l) {
        Intent detailIntent = new Intent(MainActivity.this,
                                           DetailActivity.class);
        detailIntent.putExtra("candy_name", candyList.get(i));
        startActivity(detailIntent);
    }
});
...
```

Position in the List

Passing the candy in the Intent

Getting Data from an Intent in an Activity

In the Detail Activity, we can get the Intent and then get the data stored with it.

DetailActivity.java

```
...  
Intent intent = DetailActivity.this.getIntent();  
String candyName = intent.getStringExtra("candy_name");  
...
```

Get the Intent that created this Activity

Now we can look up the data inside the Intent with the `getStringExtra()` method and the key

Products
Tropical Wave
Berry Bouncer
Grape Gummer
Apple of My Eye



If Berry Bouncer was selected, then `candy_name` will be "Berry Bouncer" 🙌

SUPER
SWEET
ANDROID
TIME



Getting Data from an Intent in an Activity

In case the Intent doesn't have that key for some reason, we don't want our program to crash. We can check that it has that extra value first.

DetailActivity.java

```
...  
Intent intent = DetailActivity.this getIntent();  
String candyName = "";  
  
if (intent.hasExtra("candy_name")) {  
    candyName = intent.getStringExtra("candy_name");  
}  
...
```

Better to be safe than sorry!

SUPER
SWEET
ANDROID
TIME



Setting the TextView's text in DetailActivity

We have the `candy_name` that was passed in, so we can set the TextView's text to that candy.

DetailActivity.java

```
...  
Intent intent = DetailActivity.this getIntent();  
String candyName = "";  
  
if (intent.hasExtra("candy_name")) {  
    candyName = intent.getStringExtra("candy_name");  
}  
  
TextView textView = (TextView) this.findViewById(R.id.text_view_name);  
textView.setText(candyName);  
...
```

First we find the TextView by its id

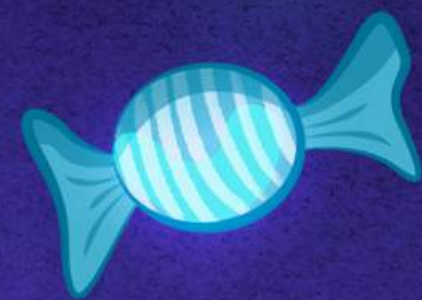
Then we can set its text

Screencast: Passing Data to the DetailActivity

SUPER
SWEET
ANDROID
TIME



SUPER SWEET ANDROID TIME



Level 2 – Section 1

Dynamic Data

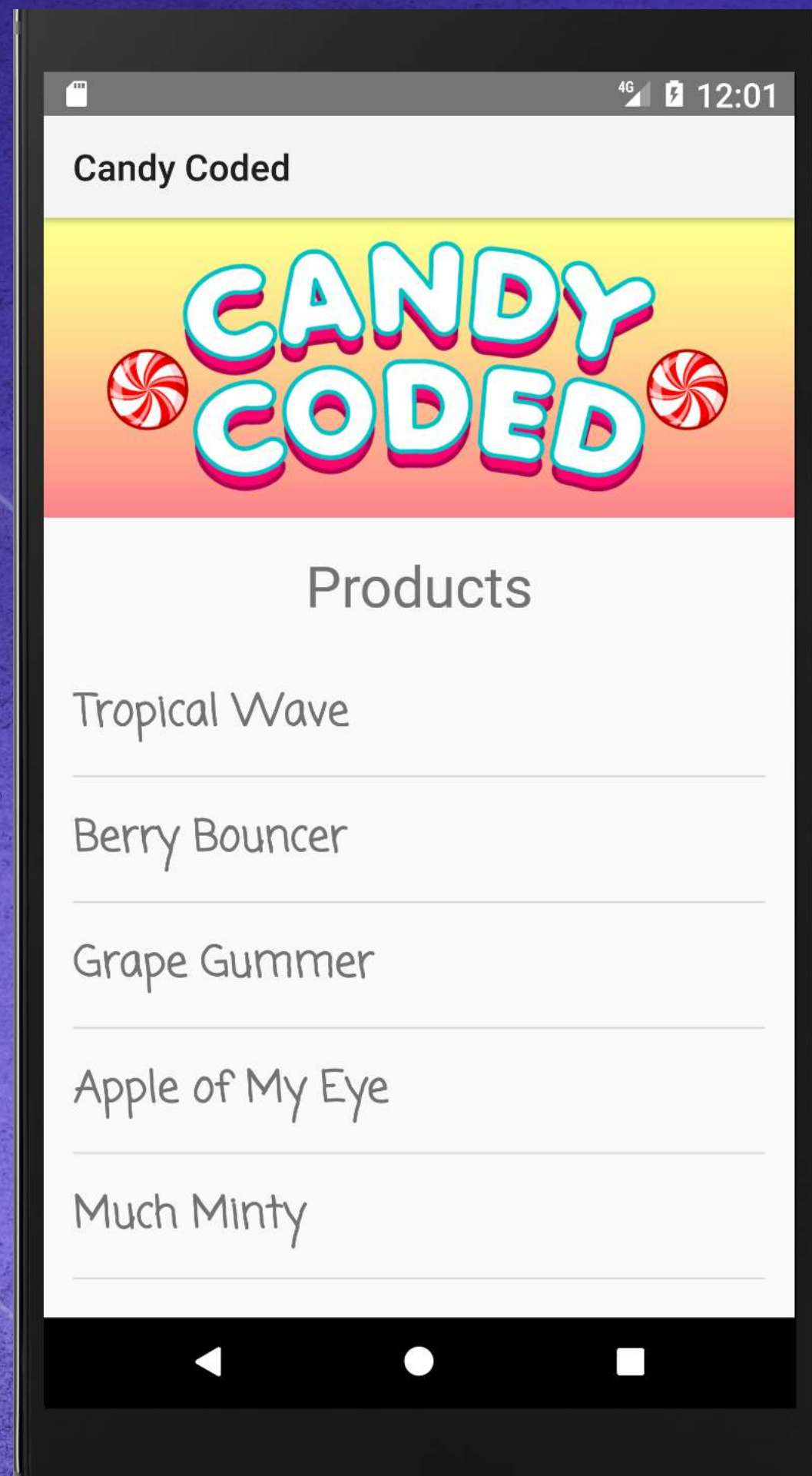
HTTP Requests with `AsyncHttpClient`

SUPER
SWEET
ANDROID
TIME



Fetching Our Candy Data Dynamically

It doesn't make sense to hard-code our store's list of candy in the app. If the candy changes, the user would have to update their app for the list to update.



Instead, we want to fetch the latest candy data with an HTTP Request from our server



Which will return the current candies as a JSON response



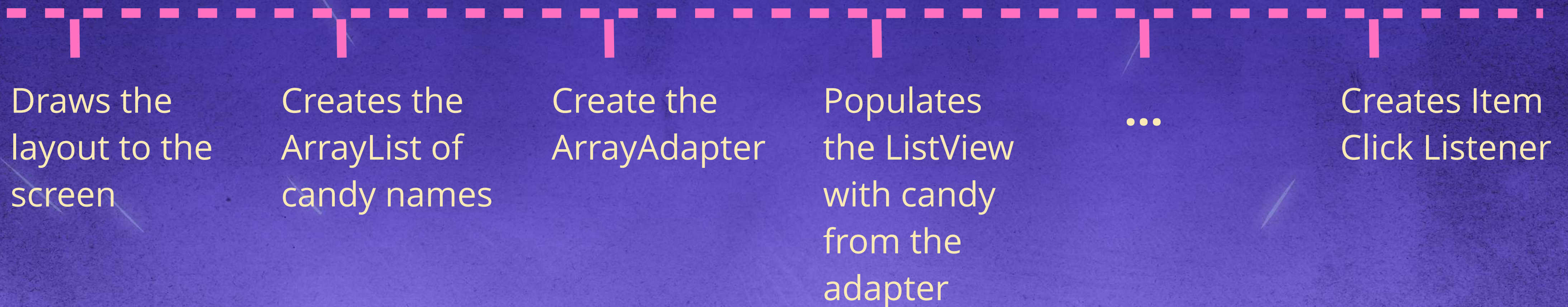
SUPER
SWEET
ANDROID
TIME



Understanding Threads

When an application is launched, a thread of execution is created for the program.

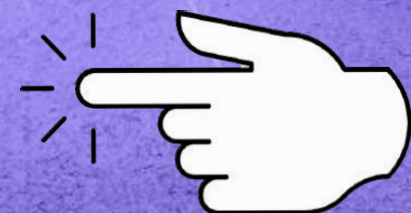
Main Thread



A program thread runs commands sequentially



The exception is callback methods, like our `onItemClicked()` method that gets called when a click triggers it



Note: In Android, the main thread handles drawing to the screen and user interaction, so it's also called the UI thread

Understanding Threads

When an application is launched, a thread of execution is created for the program.

Main Thread



If we also do time consuming operations, like network access or database queries, on the same thread, the whole UI would be blocked until those finish!

SUPER
SWEET
ANDROID
TIME



Using a Separate Background Thread

Main Thread



Instead, we can do network access or database queries on a separate background thread, that runs at the same time in the background

Background Thread



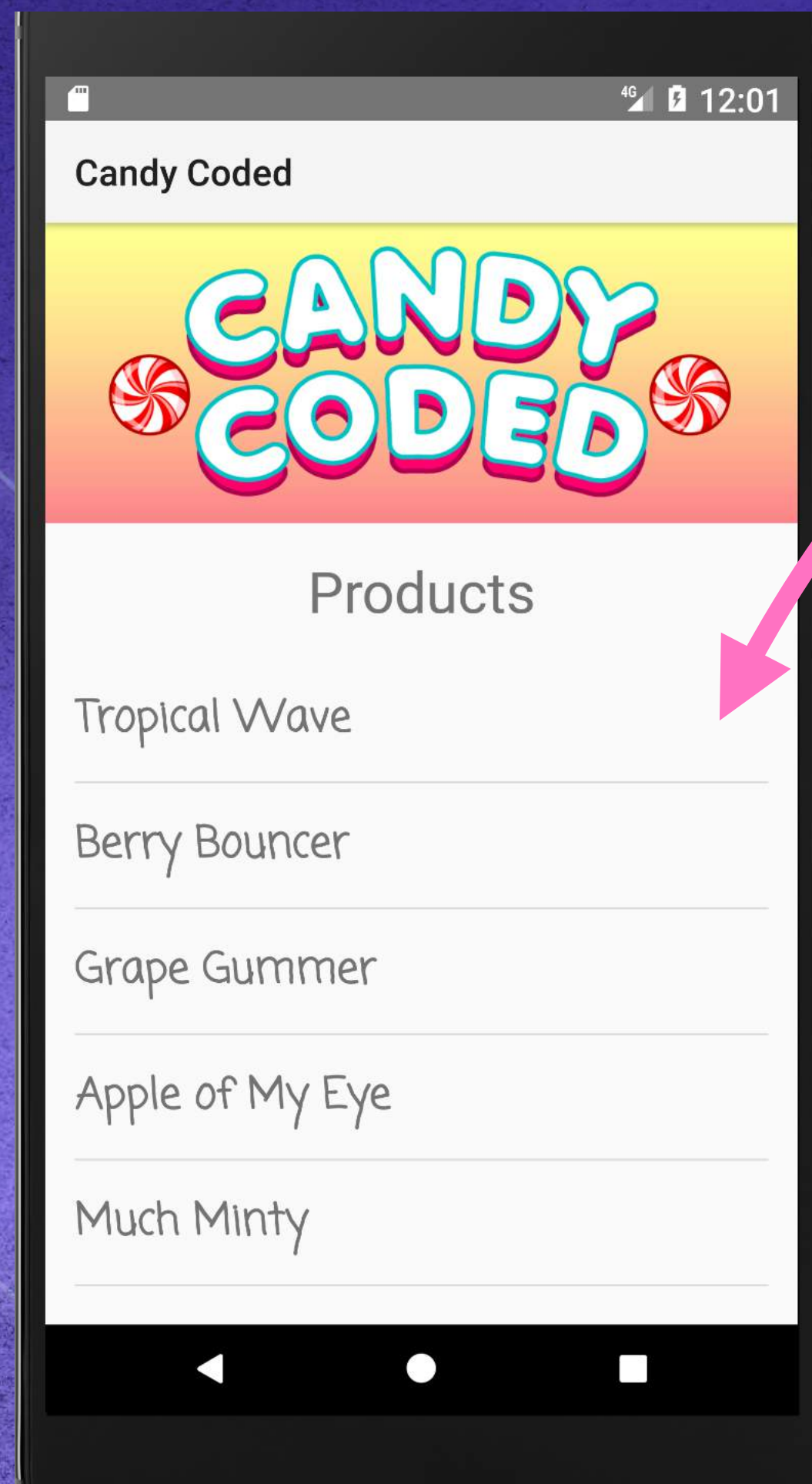
Then once that background operation finishes, it returns the results to the UI thread

This way the UI continues running smoothly!



Use AsyncHttpClient for Http Requests in the Background

Instead of writing the boilerplate code to create an Http Request in the background thread, the external Library Android Asynchronous Http Client has a class AsyncHttpClient that can do this for us.



Main UI Thread

Update the UI with the current candy items

Background Thread

AsyncHttpClient can do the HTTP Request in the background to get the current Candy items

Once AsyncHttpClient gets the result, it returns the current candy items to the UI thread



How to Use an External Library

To use an external library, like the Android Asynchronous Http Client library, we need to add it as a dependency in our build.gradle file.

build.gradle (app)

```
apply plugin: 'com.android.application'

android {
    compileSdkVersion 25 ...
    defaultConfig {
        applicationId "com.codeschool.candycoded"
        minSdkVersion 10
        targetSdkVersion 25 ...
    } ...
}

dependencies {
    ...
    compile 'com.android.support:appcompat-v7:25.3.0'
    compile 'com.android.support.constraint:constraint-layout:1.0.2'
    compile 'com.loopj.android:android-async-http:1.4.9'
}
```

Add a reference to the remote repository for the external Library to our project by adding to the bottom of our dependencies.

SUPER
SWEET
ANDROID
TIME



Using the AsyncHttpClient Library

To use the AsyncHttpClient class to make an http request, we can create a new AsyncHttpClient object and make a request with the get() method.

MainActivity.java

```
import com.loopj.android.http.*
```

```
...
```

```
public class MainActivity extends AppCompatActivity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        ...
```

```
        AsyncHttpClient client = new AsyncHttpClient();  
        client.get(...);
```

```
    }  
}
```

Create our AsyncHttpClient object

Android Studio should import this automatically, but if it doesn't we need this line

The get() method will do our request, but it has a lot in it so we'll cover that next

Using the AsyncHttpClient Library

MainActivity.java

```
AsyncHttpClient client = new AsyncHttpClient();  
client.get(  
    "https://....herokuapp.com/main/api",  
    new TextHttpResponseHandler() {  
  
        A TextHttpResponseHandler needs  
        to implement 2 callback methods -  
        onSuccess() and onFailure()  
  
    });
```

The URL:

go.codeschool.com/CandyAPI

A Response Handler object

Using the AsyncHttpClient Library

MainActivity.java

```
AsyncHttpClient client = new AsyncHttpClient();  
client.get(  
    "https://....herokuapp.com/main/api",  
    new TextHttpResponseHandler() {
```

*Inside the onSuccess() callback method
we'll just log the JSON response for now
as a debug message with Log.d()*

```
@Override  
public void onSuccess(int statusCode, Header[] headers,  
    String response) {  
    Log.d("AsyncHttpClient", "response = " + response);  
}
```

```
});
```


Using the AsyncHttpClient Library

MainActivity.java

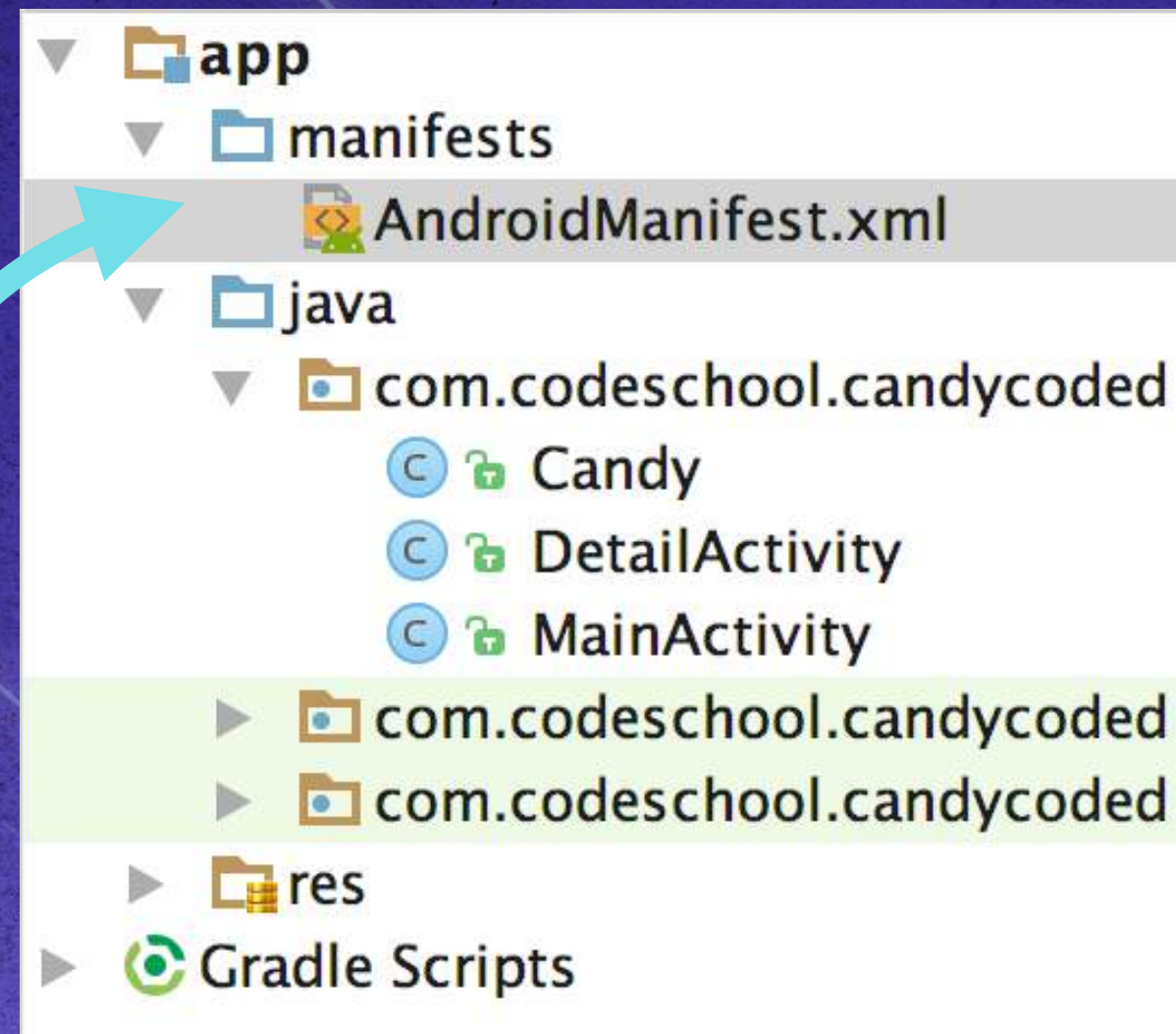
```
AsyncHttpClient client = new AsyncHttpClient();
client.get(
    "https://....herokuapp.com/main/api",
    new TextHttpResponseHandler() {
        @Override
        public void onFailure(int statusCode, Header[] headers,
                               String response, Throwable throwable) {
            Log.e("AsyncHttpClient", "response = " + response);
        }

        @Override
        public void onSuccess(int statusCode, Header[] headers,
                               String response) {
            Log.d("AsyncHttpClient", "response = " + response);
        }
    });
```

*For a failure
we'll log the
same thing
but with
Log.e() for
errors*

Allowing Internet Access in the Manifest

The manifest file provides essential information about your app to the Android system.



Every application must have an `AndroidManifest.xml` file (with precisely that name) in its root directory

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<manifest xmlns:android="..."
    package="com.codeschool.candycoded">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/Theme.AppCompat.Light">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name=
                    "android.intent.action.MAIN" />
                ...
            </intent-filter>
        </activity>
    </application>
</manifest>
```


Allowing Internet Access in the Manifest

We'll add the Internet permission to the top of the Manifest file.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="..."
    package="com.codeschool.candycoded">
    <uses-permission android:name="android.permission.INTERNET" />
    <application ...
</manifest>
```

*<uses-permission> requests a permission that the app needs.
Permissions are granted by the user while the app is running*

SUPER
SWEET
ANDROID
TIME



Screencast: Logging Debug Messages

SUPER
SWEET
ANDROID
TIME



Level 2 – Section 2

Dynamic Data

Parsing JSON

SUPER
SWEET
ANDROID
TIME



How to Parse a Simple JSON String

The JSON the Candy API returns has more information, but let's start with a simple example that only has the name of 1 candy.

```
String response = "[  
  {  
    "name": "Tropical Wave",  
  }  
]"
```

*A JSONArray
is inside []*

*A JSONObject
is inside {}*

Each value has a key

```
JSONArray candyArray = new JSONArray(candyJsonStr);  
JSONObject jsonObj = candyArray.getJSONObject(0);  
String candyName = jsonObj.getString("name");
```

After this code runs, **candyName** *is* **"Tropical Wave"**

SUPER
SWEET
ANDROID
TIME



The Actual JSON Response Has More Information

The JSON the Candy API actually returns looks like this.

A JSONArray
is inside []

A JSONObject
is inside {}

Keys for each
property

```
[
  {
    "id": 1,
    "name": "Tropical Wave",
    "image": "https://s3.amazonaws.com/.../gumdrops-1.png",
    "price": "5.99",
    "description": "These tropical-flavored gummies ...",
  },
  {
    "id": 2,
    "name": "Berry Bouncer",
    "image": "https://s3.amazonaws.com/.../gumdrops-2.png",
    "price": "4.99",
    "description": "Berry delicious! This ...",
  },
  ...
]
```

*Since we have multiple properties for each Candy,
we want a way to group these together...*

Storing the Candy Data in a Candy Class

We want to group the candy properties - name, image, price **and** description - **together** as one object. We can do that by creating a Candy class.

JSON data
for each
Candy

```
{  
  "id": 1,  
  "name": "Tropical Wave",  
  "image": "https://s3.amazonaws.com/.../gumdrops-1.png",  
  "price": "5.99",  
  "description": "These tropical-flavored gummies ...",  
}
```

Candy
class

```
public class Candy {  
  public int id;  
  public String name;  
  public String imageURL;  
  public String price;  
  public String description;  
}
```

Let's map JSON data
to a Java class

SUPER
SWEET
ANDROID
TIME



Screencast: How to Create a New Class

SUPER
SWEET
ANDROID
TIME



Using the GSON Library to Parse the JSON for Us

We could write Java code to loop over every JSONObject in the JSONArray and get each Candy's properties into this class, but the GSON Library will do this for us.

*JSON data
for each
Candy*

```
{  
  "id": 1,  
  "name": "Tropical Wave",  
  "image": "https://s3.amazonaws.com/.../gumdrops-1.png",  
  "price": "5.99",  
  "description": "These tropical-flavored gummies ..."  
}
```

Candy class

```
public class Candy {  
  public int id;  
  public String name;  
  public String image;  
  public String price;  
  public String description;  
}
```

*The GSON Library can automatically
convert JSON objects into Candy
objects for us!*

*For GSON to work without any other
configuration, the property names in
your class need to match up exactly
with the keys in your JSON file.*

Adding the GSON Library as a Dependency

To use the GSON Library, we need to add it as a dependency in our `build.gradle` file.

`build.gradle (app)`

```
apply plugin: 'com.android.application'

android {
    compileSdkVersion 25 ...
    defaultConfig {
        applicationId "com.codeschool.candycoded"
        minSdkVersion 10
        targetSdkVersion 25 ...
    } ...
}

dependencies {
    ...
    compile 'com.loopj.android:android-async-http:1.4.9'
    compile 'com.google.code.gson:gson:2.8.0'
}
```

Add a reference to the GSON Library to our project by adding to the bottom of our dependencies

SUPER
SWEET
ANDROID
TIME



How to Use GSON

The GSON Library converts JSON strings to Java objects. But note that this will only work if your class properties match the JSON keys, otherwise GSON requires customization.

```
String response = "{
    \"id\": 1,
    \"name\": \"Tropical Wave\",
    \"image\": \"https://.../gumdrops-1.png\",
    \"price\": \"5.99\",
    \"description\": \"These tropical-flavored gummies ...\"
}";

Gson gson = new GsonBuilder().create();
Candy candy = gson.fromJson(response, Candy.class);
```

After this code is run all of candy's properties are set:

<code>candy.name</code>	is	<code>"Tropical Wave"</code>
<code>candy.price</code>	is	<code>5.99</code>

etc.

The JSON string

The class we want to convert to

SUPER
SWEET
ANDROID
TIME



How to Use GSON for an Array of Objects

```
String response = "[{"id": 1,
                    "name": "Tropical Wave",
                    "image": "https://.../gumdrops-1.png",
                    "price": "5.99",
                    "description": "These tropical-flavored gummies ..."},
                    {"id": 2,
                    "name": "Tropical Wave",
                    "image": "https://.../gumdrops-1.png",
                    "price": "5.99",
                    "description": "These tropical-flavored gummies ..."},
                    ...
                    ]";
```

```
Gson gson = new GsonBuilder().create();
Candy[] candies = gson.fromJson(response, Candy[].class);
```

We can just specify we have an Array of Candy objects

Adding our GSON Code to Our AsyncHttpClient Request

We'll add our GSON code to create an Array of Candy objects after we get our JSON response.

MainActivity.java

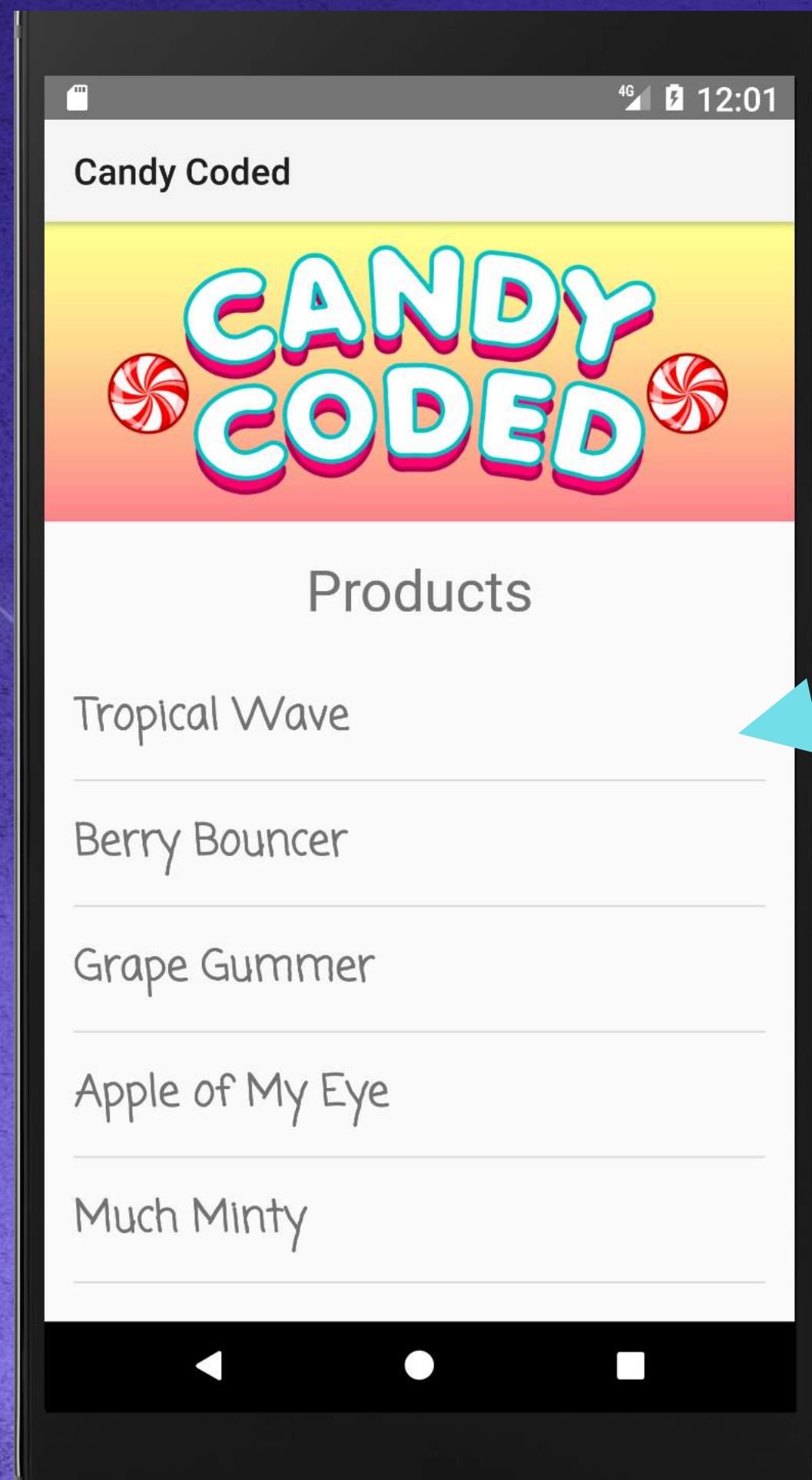
```
AsyncHttpClient client = new AsyncHttpClient();
client.get(
    "https://....herokuapp.com/main/api",
    new TextHttpResponseHandler() {
        @Override public void onFailure(...) {}

        @Override public void onSuccess(int status, Header[] headers,
                                         String response) {
            Log.d("AsyncHttpClient", "response = " + response);
            Gson gson = new GsonBuilder().create();
            Candy[] candies = gson.fromJson(response, Candy[].class);
        }
    });
```

*Now we have all our Candy objects, let's
update the adapter to use the Candy names.*

Adding the Candy Names to the Adapter

Now we have all our Candy objects, let's update the adapter to use the Candy names.



candies

We can put each Candy's name in our ListView by adding each candy.name to the Adapter.

SUPER
SWEET
ANDROID
TIME



Adding our Candy Names to Our ListView's Adapter

MainActivity.java

```
AsyncHttpClient client = new AsyncHttpClient();
client.get(
    "https://....herokuapp.com/main/api",
    new TextHttpResponseHandler() {
        @Override public void onFailure(...) {}

        @Override public void onSuccess(int status, Header[] headers,
                                         String response) {
            Log.d("AsyncHttpClient", "response = " + response);
            Gson gson = new GsonBuilder().create();
            Candy[] candies = gson.fromJson(response, Candy[].class);

            adapter.clear();
            for(Candy candy : candies) {
                adapter.add(candy.name);
            }
        }
    });
```

First clear what's in the adapter now

Then add each candy's name

Screencast: The ListView Updating in the Background

SUPER
SWEET
ANDROID
TIME



Level 2 – Section 3

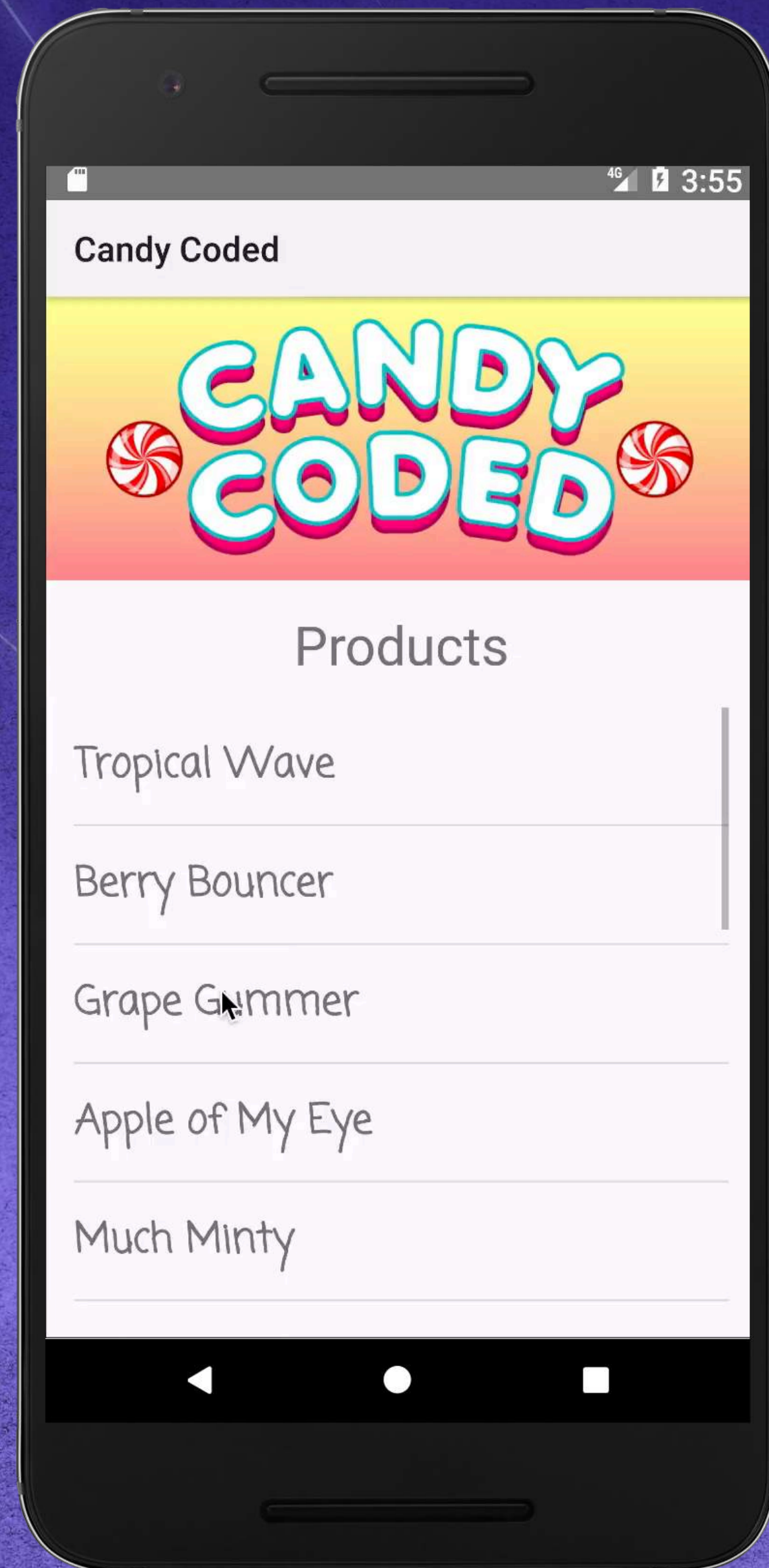
Dynamic Data

Passing Dynamic Data to the Detail Activity

SUPER
SWEET
ANDROID
TIME



Pass Live Data to the Detail Activity



Right now our Detail Activity is only showing the candy name

We also want to display the image, price and description we got from the Candy API

To do that we'll also need to pass the image, price and description from the array of Candy objects and send it to the Detail Activity

SUPER
SWEET
ANDROID
TIME



We Want to Pass the Candy Values with the DetailActivity Intent

We also want to pass the Candy's image URL, price, and description.

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private Candy[] candies; We also want to use the Candy's array throughout  
    ...                               the entire class, so we'll declare it here.  
    @Override protected void onCreate(Bundle savedInstanceState) {  
        ...  
        listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {  
            @Override  
            public void onItemClick(AdapterView<?> ...) {  
                Intent detailIntent = new Intent(this, DetailActivity.class);  
                detailIntent.putExtra("candy_name", candy_list.get(i));  
                This is where we want to pass the Candy's image URL, price, and  
                description to the Detail Activity  
                startActivity(detailIntent);  
            }  
        });  
    }  
    ...  
}
```


Passing the Candy Values to the DetailActivity Intent

We can pass the Candy's image URL, price, and description with the `putExtra()` method.

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    private Candy[] candies;
    ...
    @Override protected void onCreate(Bundle savedInstanceState) {
        ...
        listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> ...) {
                Intent detailIntent = new Intent(this, DetailActivity.class);
                detailIntent.putExtra("candy_name", candies[i].name);
                detailIntent.putExtra("candy_image", candies[i].imageURL);
                detailIntent.putExtra("candy_price", candies[i].price);
                detailIntent.putExtra("candy_desc", candies[i].description);
                startActivity(detailIntent);
            }
        });
        ...
    }
}
```


Getting the Data from the Intent in DetailActivity

Back in the DetailActivity class, we can get the image, price, and description after we displayed the candy name in the textView and then display them.

DetailActivity.java

```
...  
Intent intent = DetailActivity.this getIntent();  
String candyName = "";  
  
if (intent.hasExtra("candy_name")) {  
    candyName = intent.getStringExtra("candy_name");  
}  
  
TextView textView = (TextView) this.findViewById(R.id.name_text_view);  
textView.setText(candyName);  
...  
  
This is where we can display the image, price and description
```

First, let's Log the image URL, price and description to make sure they're coming in correctly

Logging the DetailActivity Intent's Data

After we get our values with `getStringExtra()`, we'll log all of the values to make sure we got them correctly before we worry about displaying them in the layout.

DetailActivity.java

```
...  
String candyImage = intent.getStringExtra("candy_image");  
String candyPrice = intent.getStringExtra("candy_price");  
String candyDesc = intent.getStringExtra("candy_desc");  
  
Log.d("DetailActivity", "Intent data: " + candyImage + ", " +  
      candyPrice + ", " + candyDesc);  
...
```

We can get our passed in values from the Intent with the same `getStringExtra()` method we used for the name

Then we'll log these properties with `Log.d()` to make sure they look correct

Logging the DetailActivity Intent's Data

We also want to add some checks to make sure these values actually exist to prevent errors.

DetailActivity.java

```
...  
String candyImage = "";  
if (intent.hasExtra("candy_image"))  
    candyImage = intent.getStringExtra("candy_image");
```

```
String candyPrice = 0;  
if (intent.hasExtra("candy_price"))  
    candyPrice = intent.getStringExtra("candy_price");
```

```
String candyDesc = "";  
if (intent.hasExtra("candy_desc"))  
    candyDesc = intent.getStringExtra("candy_desc");
```

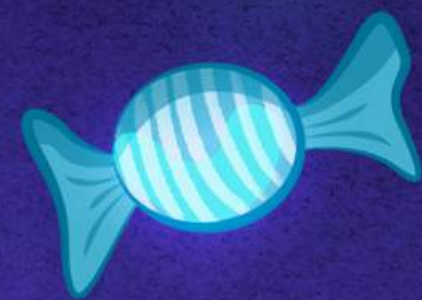
```
Log.d("DetailActivity", "Intent data: " + candyImage + ", " +  
    candyPrice + ", " + candyDesc);  
...
```


Screencast: Logging the DetailActivity Intent's Data

SUPER
SWEET
ANDROID
TIME



SUPER
SWEET
ANDROID
TIME



Level 3 – Section 1

The Detail Activity Layout

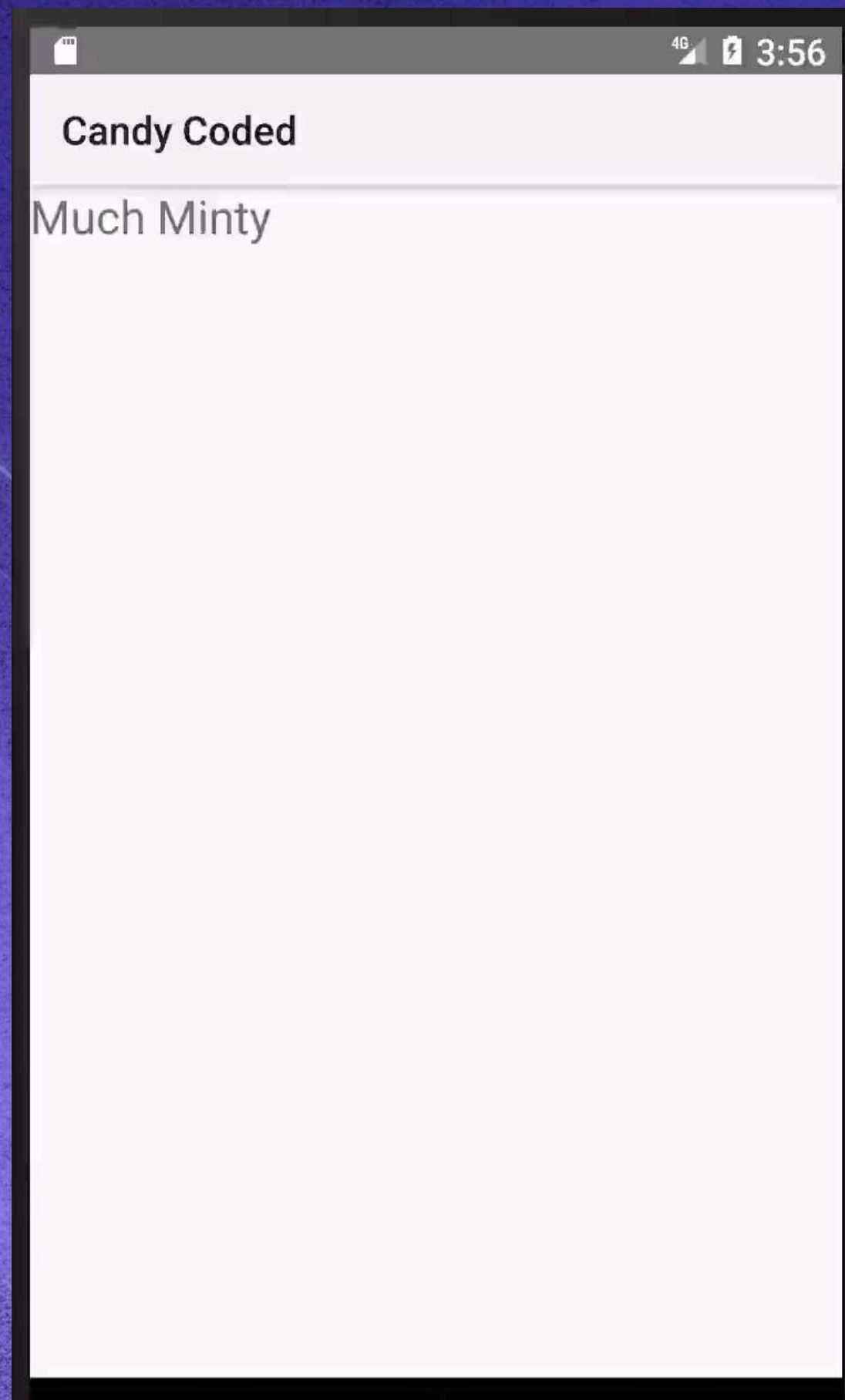
The Constraint Layout

SUPER
SWEET
ANDROID
TIME

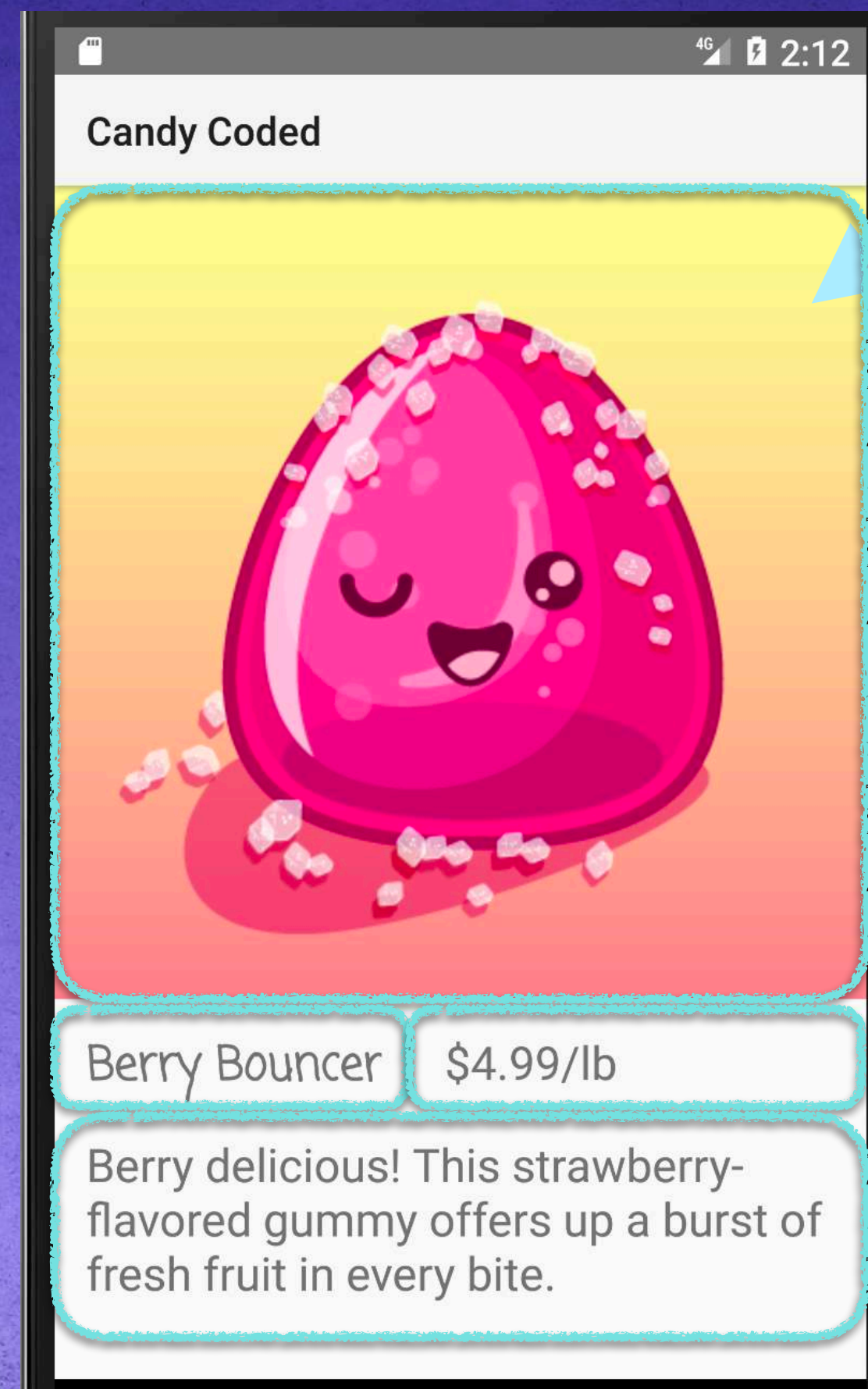


What We Want DetailActivity's Layout to Look Like

Right now the Detail Activity is only showing the Candy Name, we want to add TextView's for the price and description and an ImageView for the image.



Our Detail Activity right now



Our final Detail Activity

*We can use a
ConstraintLayout to
arrange all of these
Views relative to each
other*



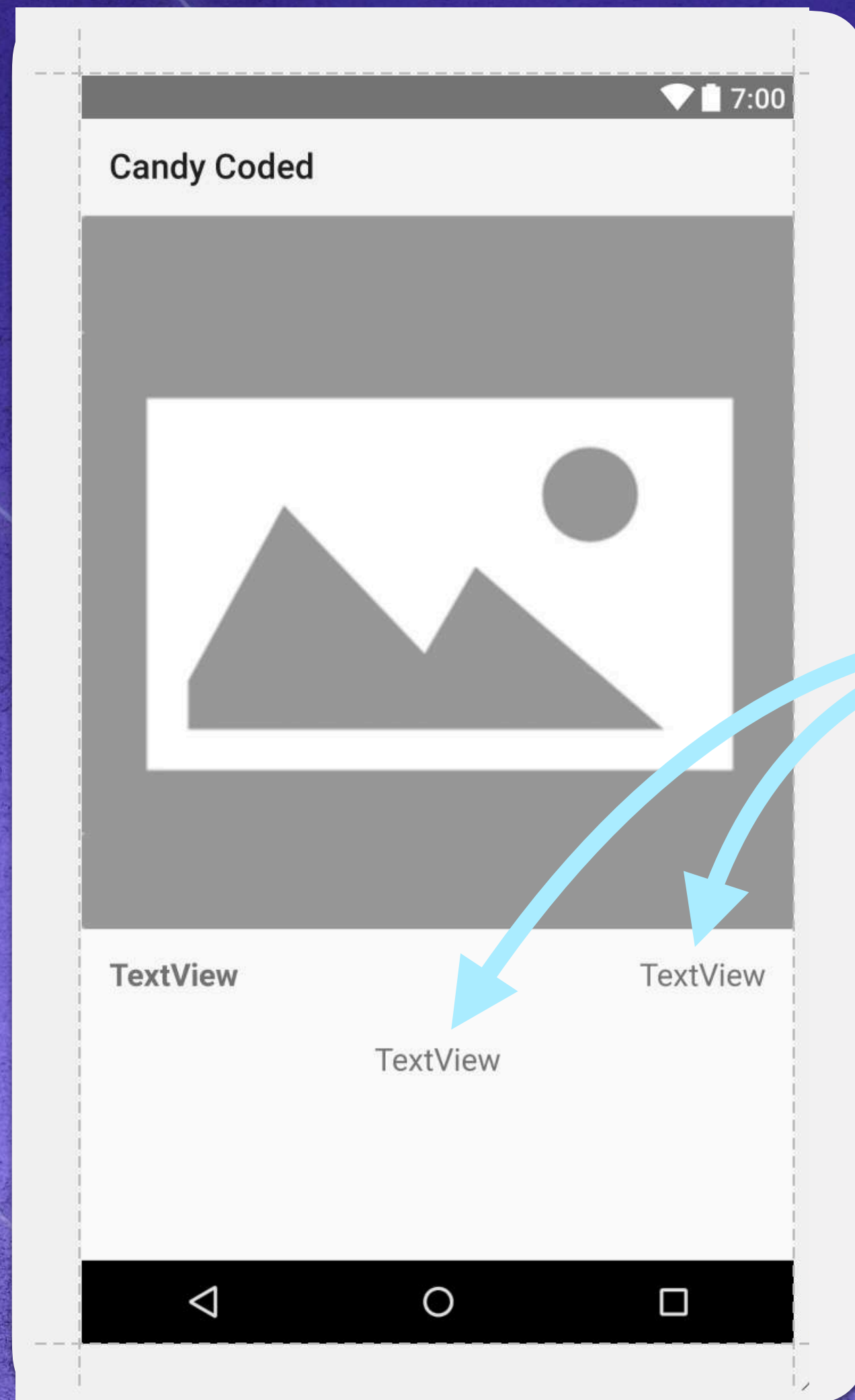
Screencast: Configuring the DetailActivity's ConstraintLayout

SUPER
SWEET
ANDROID
TIME



Our New Detail Activity Layout

Now that we have our new Detail Activity's layout in place, let's edit the Java code to fill the Views with data.



*Let's write the code to fill
the price and description
TextViews with actual
data*

SUPER
SWEET
ANDROID
TIME



Setting the TextView' text in DetailActivity

In DetailActivity, after we set the candy's name we can set the text for the other TextView objects.

DetailActivity.java

```
...
public class DetailActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        ...
        Intent intent = DetailActivity.this getIntent();
        String candyName = "";
        if (intent != null && intent.hasExtra("candy_name")) {
            candyName = intent.getStringExtra("candy_name");
        }
        TextView textViewName = (TextView) this.findViewById(
                                                    R.id.text_view_name);
        textViewName.setText(candyName);
        Let's set the text for the other TextViews
        here using the same 3 steps as above
    }
}
```

1 Get the value sent with the intent.

2 Find the TextView

3 Set the TextView's text

Setting the Description TextViews' text

Setting the text for the description TextView will be similar to setting it for the name.

DetailActivity.java

```
...
public class DetailActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        ...

        String candyDesc = "";
        if (intent != null && intent.hasExtra("candy_desc")) {
            candyDesc = intent.getStringExtra("candy_desc");
        }

        TextView textViewDesc = (TextView)this.findViewById(
            R.id.text_view_desc);
        textViewDesc.setText(candyDesc);

    }
}
```

Get the value sent with the intent

Find the TextView

Set the TextView's text

Setting the Price TextViews' text

Setting the text for the description TextView will also be similar to setting it for the name.

DetailActivity.java

```
...
public class DetailActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        ...

        String candyPrice = "";
        if (intent != null && intent.hasExtra("candy_price")) {
            candyPrice = intent.getStringExtra("candy_price");
        }

        TextView textViewPrice = (TextView) this.findViewById(
            R.id.text_view_price);
        textViewPrice.setText(candyPrice);

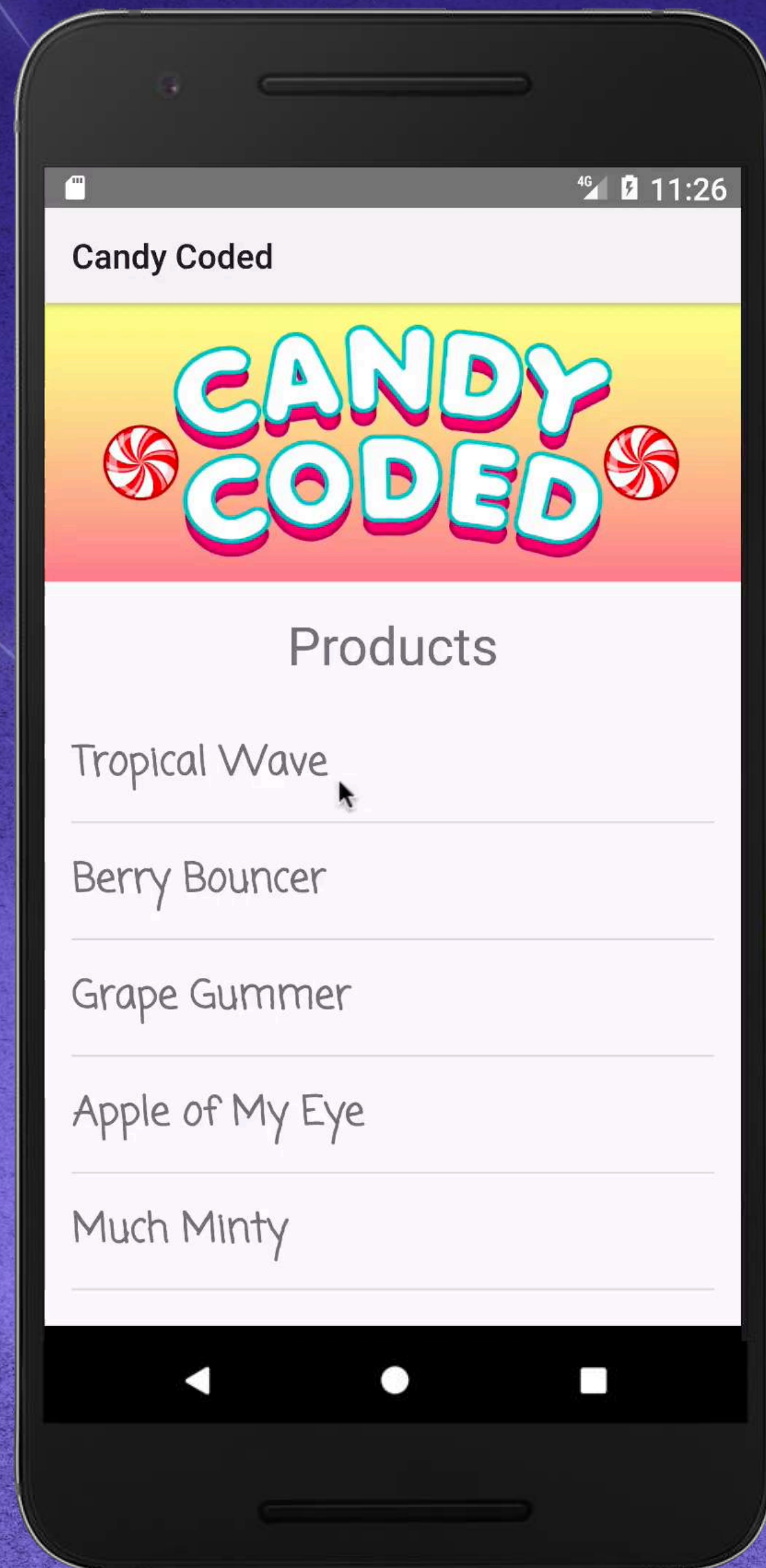
    }
}
```

Get the value sent with the intent.

Find the TextView

Set the TextView's text

Demo: The TextViews Have Live Data



Now our TextViews have live data!

Next we'll add our image!

SUPER
SWEET
ANDROID
TIME



Level 3 – Section 2

The Detail Activity Layout

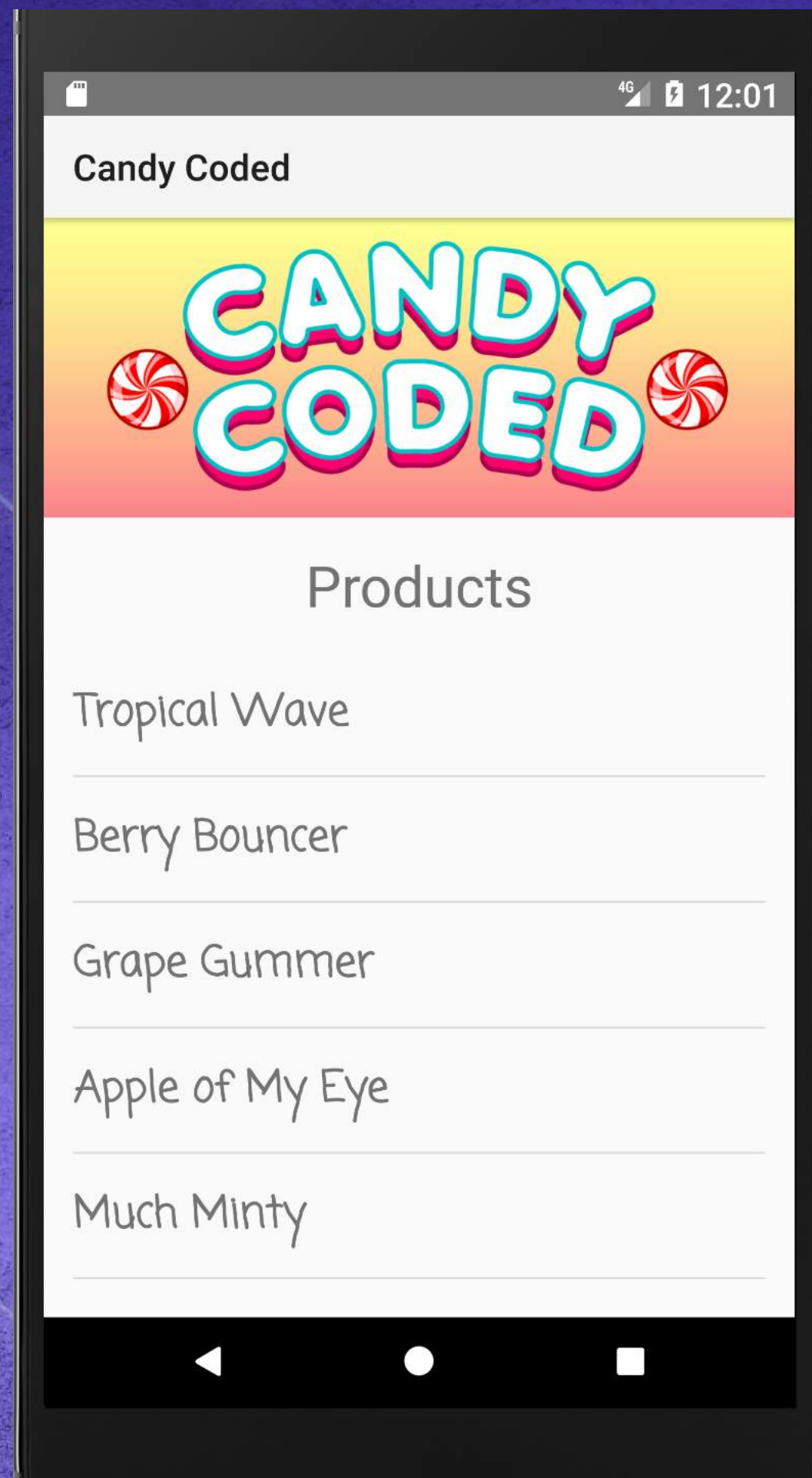
Using the Picasso Image Library

SUPER
SWEET
ANDROID
TIME



Displaying an Image via a URL in an ImageView

Instead of writing somewhat lengthy code to download images into our app and cache them, Picasso is an Android library we can use to do this for us.



Web
Server



SUPER
SWEET
ANDROID
TIME



Caching

We need caching because getting files from a distant web server takes much longer than getting locally cached images from memory.



Adding the Picasso Library to our Project

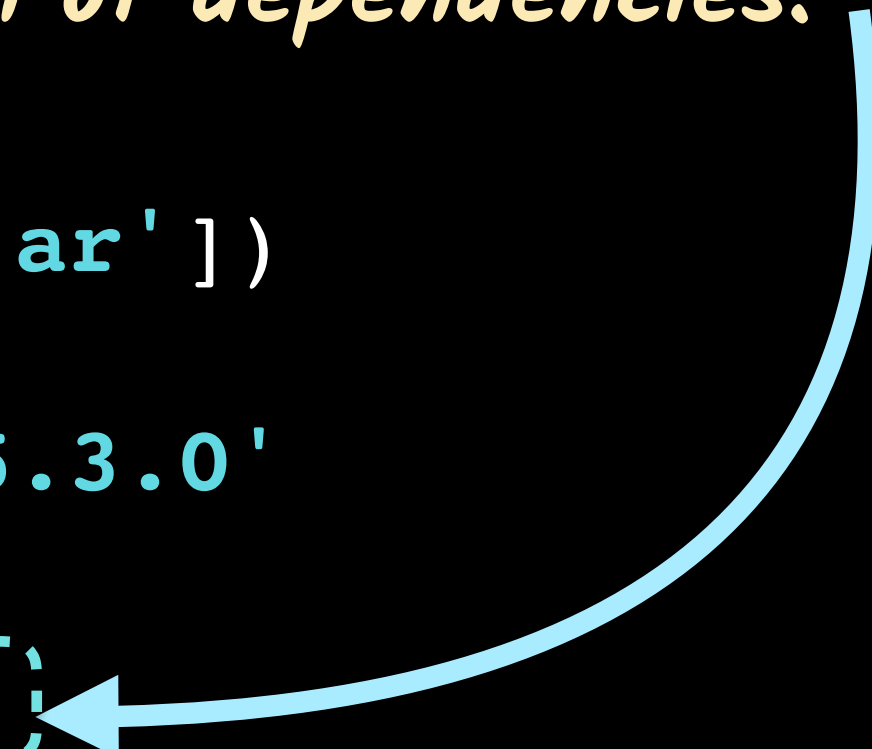
build.gradle (app)

```
apply plugin: 'com.android.application'

android {
    compileSdkVersion 25
    buildToolsVersion "25.0.0"
    defaultConfig {
        applicationId "com.codeschool.candycoded"
        minSdkVersion 10
        targetSdkVersion 25 ...
    } ...
}

dependencies {
    compile fileTree(dir: 'libs', include: ['*.jar'])
    androidTestCompile(...)
    compile 'com.android.support:appcompat-v7:25.3.0'
    ...
    compile 'com.squareup.picasso:picasso:2.5.2'
}
```

Add the Picasso Library to our project by adding to the bottom of dependencies.



Then when we sync our project, gradle will automatically download and compile any dependencies.

Using Picasso to Load an Image into an ImageView

Now that Picasso has been included in our project, it's really easy to use it in our code.

```
Picasso.with(this).load(candy_image).into(imageView);
```

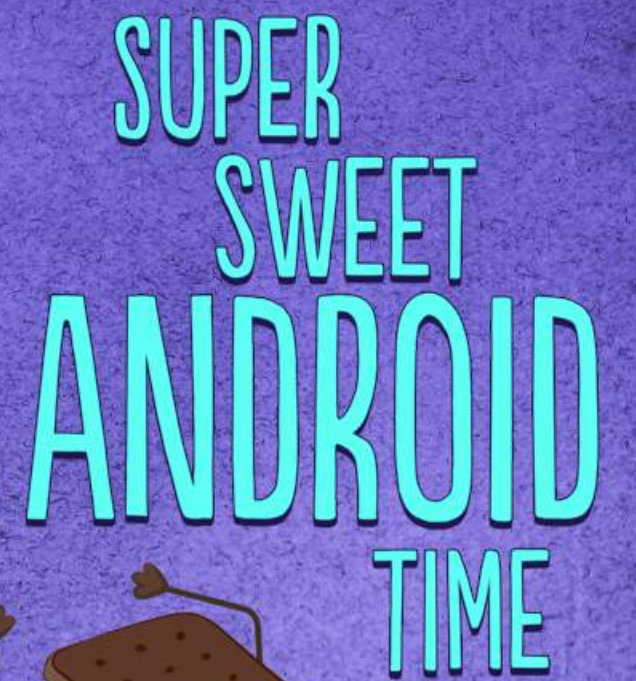
The context, this means that we're going to display the image in this DetailActivity

The url where Picasso will download the image from

The ImageView where we're going to display the image

Remember by using Picasso, Android Studio will automatically add the following import line to the top of your file. Or you can add it yourself.

```
import com.squareup.picasso.Picasso;
```



Using Picasso to Load an Image into an ImageView

DetailActivity.java

```
...
Intent intent = DetailActivity.this.getIntent();
...
String candy_image = "";
if (intent != null && intent.hasExtra("candy_image")) {
    candy_image = intent.getStringExtra("candy_image");
}

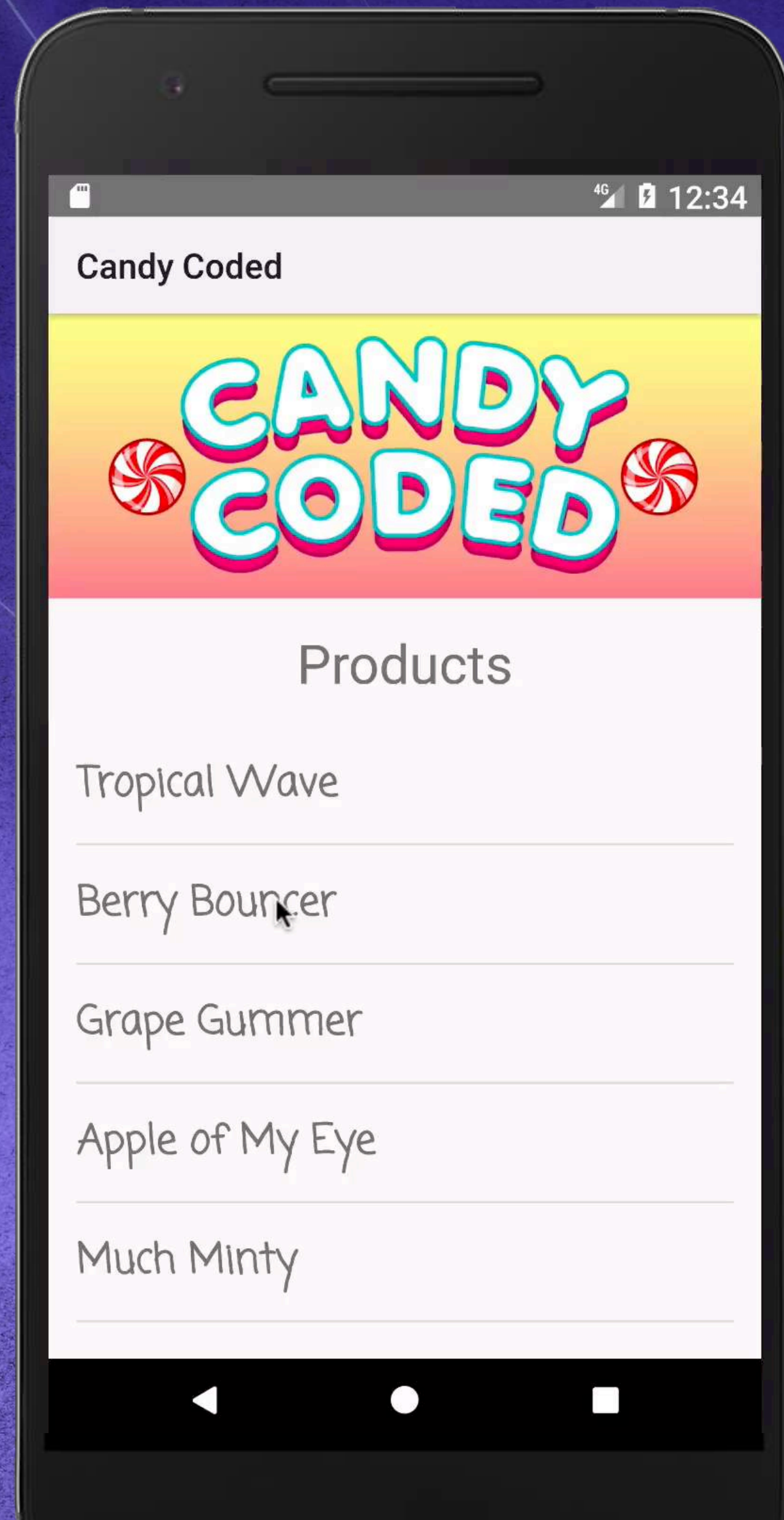
ImageView imageView = (ImageView)this.findViewById(
    R.id.image_view_candy);
Picasso.with(this).load(candy_image).into(imageView);
...
```

*First we need the image url
passed in with the Intent*

*Then we need to find
our ImageView*

*We can then use Picasso to load and cache our image
from the URL and display it in our ImageView*

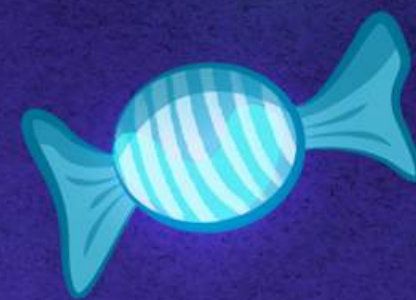
Picasso in Action



Now we can see Picasso loading our images dynamically from the image URL!



SUPER
SWEET
ANDROID
TIME



Level 4 – Section 1

Storing Data

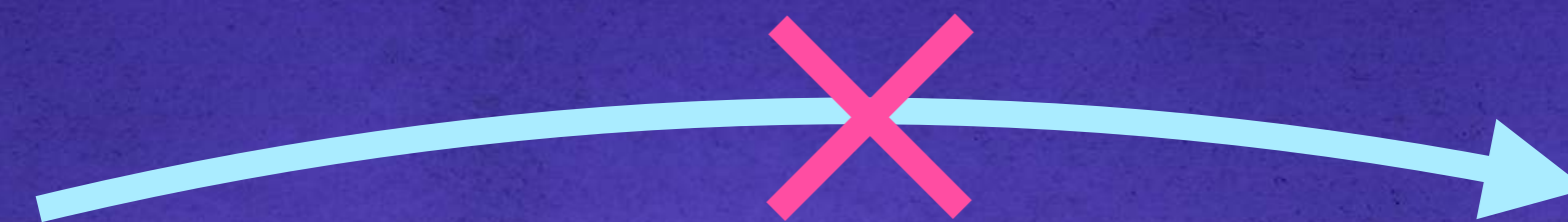
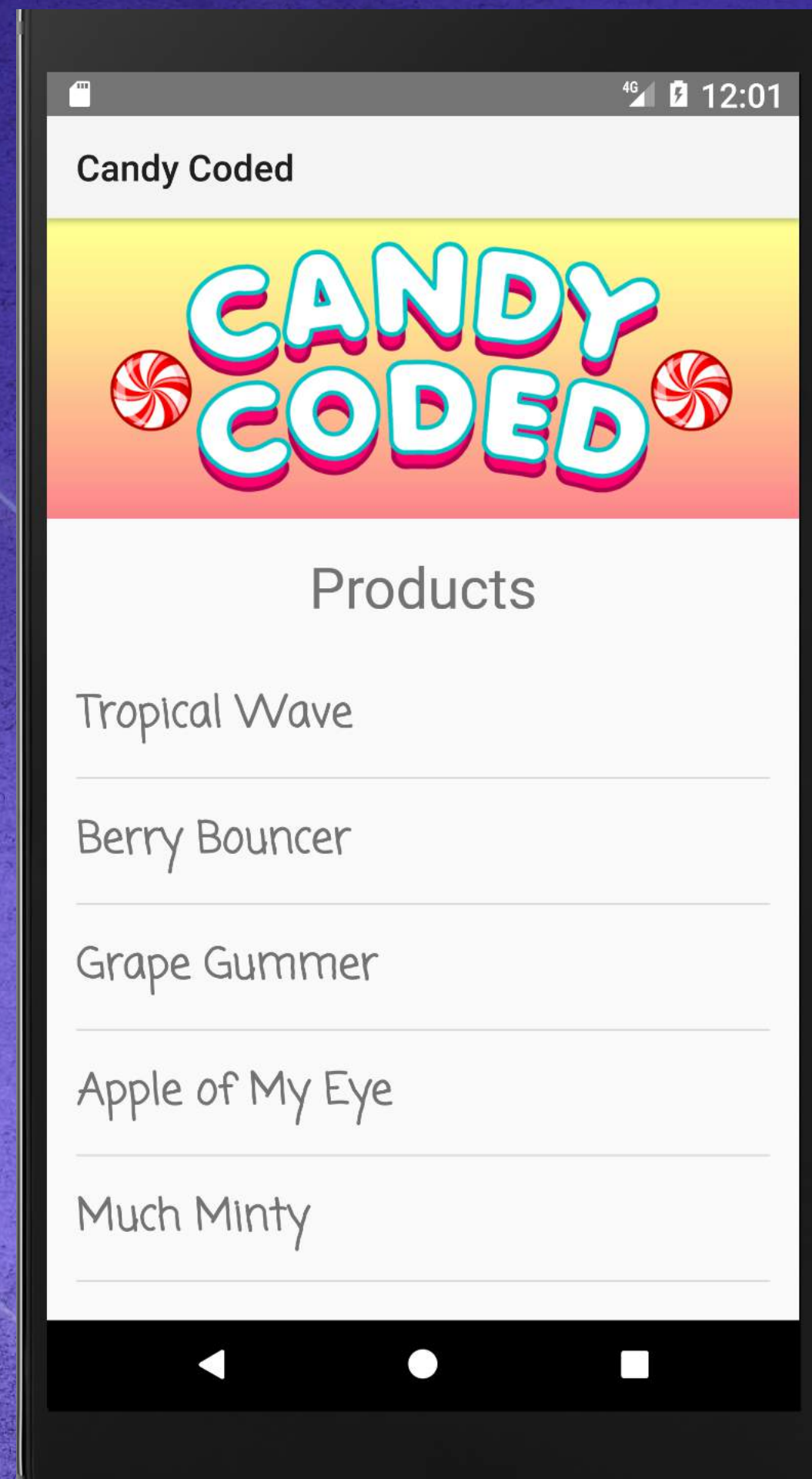
Using a SQLite Database

SUPER
SWEET
ANDROID
TIME



Storing Candy Data in Our App

Why do we want to store Candy data in our app? Can't we always just do a network request?



Sometimes you want data offline



You can store app data on your device and read/write without an Internet connection



Different Ways of Storing Data in Android Apps

A SharedPreferences **Object** or a SQLiteDatabase **are common ways** to store data in Android apps.

Shared Preferences

Stores small amounts of data in key/value pairs



: “Location”

Value: “Orlando”

SQLite Database

Stores structured data like our Candy data



Name	Price	Description	Image
Much Minty	4.50	This peppermint ...	...
So Fresh	5.50	The wintergreen ...	...
Uni-Pop	9.99	The sugary magic...	...
...			

Database Operations

To use a database we need to write some SQL commands to create a table and eventually perform operations like inserting, reading and deleting.



Name	Price	Description	Image
Much Minty	4.50	This peppermint ...	...
So Fresh	5.50	The wintergreen ...	...
Uni-Pop	9.99	The sugary magic...	...
...			

We can write SQL for creating a table in a Contract class



*We'll cover all of the database concepts you need for this course
If you want learn more database concepts, check out our Try SQL Course!*



Defining a Contract Class

Just like defining a SQL database's organization in a schema, a Contract class is the place to define your database's structure in an Android app



We are going to create a Contract class to define constants like the table and column names.

Candy Contract

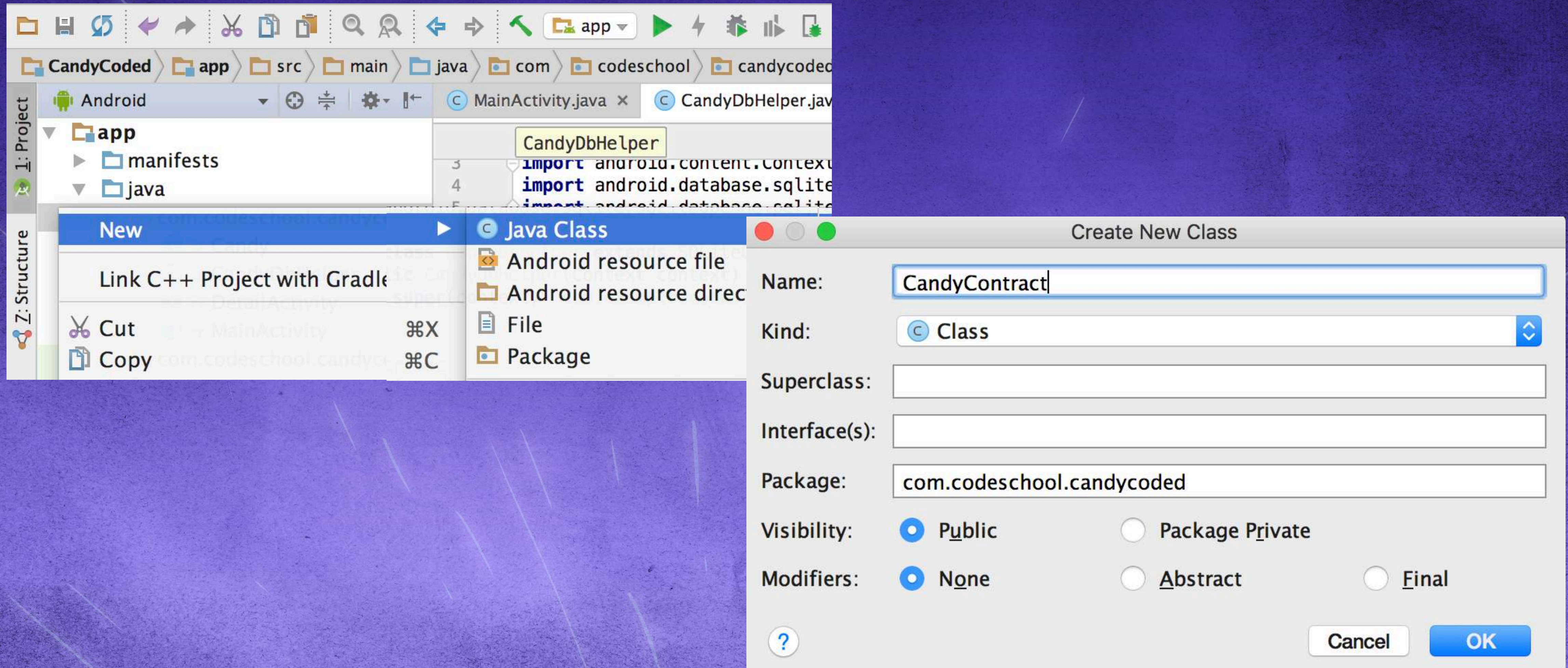
It can also act like a greeting card for other apps to know what data is provided.

SUPER
SWEET
ANDROID
TIME



Creating our CandyContract Class

We will create a new Java class called CandyContract.



The CandyContract Class

We will define our database name, database version, table name, and column names in our CandyContract **class**.

CandyContract.java

```
public class CandyContract {  
    public static final String DB_NAME = "candycoded.db";  
    public static final int DB_VERSION = 1;  
  
    public static class CandyEntry {  
  
        Then create an inner class for each table that lists its columns.  
  
    }  
}
```

A good rule is to put definitions that are global to your whole database in the root level of the class

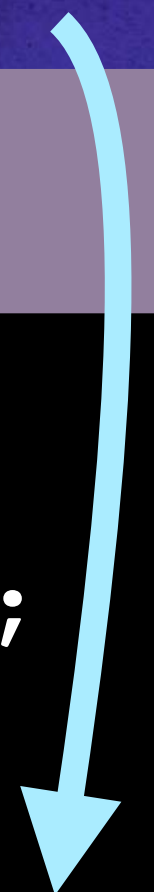
The CandyContract Class

By implementing the BaseColumns Interface, your inner class can inherit a primary key field called `_ID` that we'll need later

CandyContract.java

```
public class CandyContract {
    public static final String DB_NAME = "candycoded.db";
    public static final int DB_VERSION = 1;

    public static class CandyEntry implements BaseColumns {
        public static final String TABLE_NAME = "candy";
        public static final String COLUMN_NAME_NAME = "name";
        public static final String COLUMN_NAME_PRICE = "price";
        public static final String COLUMN_NAME_DESC = "description";
        public static final String COLUMN_NAME_IMAGE = "image";
    }
}
```



Creating a Database Table with Raw SQL

The Create Table command creates a table in our database with the specified columns and data types.

```
CREATE TABLE candy  
(  
  _ID INTEGER PRIMARY KEY,  
  Name TEXT,  
  Price TEXT,  
  Description TEXT,  
  Image TEXT  
)
```

*This SQL command would generate
this table structure*

_ID	Name	Price	Description	Image



Adding a Statement to Create the Candy Table

Once we have defined how our database looks, we can create SQL statements that create and maintain the database and tables.

CandyContract.java

```
public class CandyContract {  
    public static final String DB_NAME = "candycoded.db";  
    public static final int DB_VERSION = 1;  
  
    public static final String SQL_CREATE_ENTRIES =  
        "CREATE TABLE " + CandyEntry.TABLE_NAME + " (" +  
        CandyEntry._ID + " INTEGER PRIMARY KEY," +  
        CandyEntry.COLUMN_NAME_NAME + " TEXT," +  
        CandyEntry.COLUMN_NAME_PRICE + " TEXT," +  
        CandyEntry.COLUMN_NAME_DESC + " TEXT," +  
        CandyEntry.COLUMN_NAME_IMAGE + " TEXT)";  
  
    public static class CandyEntry implements BaseColumns {  
        ...  
    }  
}
```


Adding a Statement to Drop the Candy Table

Once we have defined how our database looks, we can create SQL statements that create and maintain the database and tables.

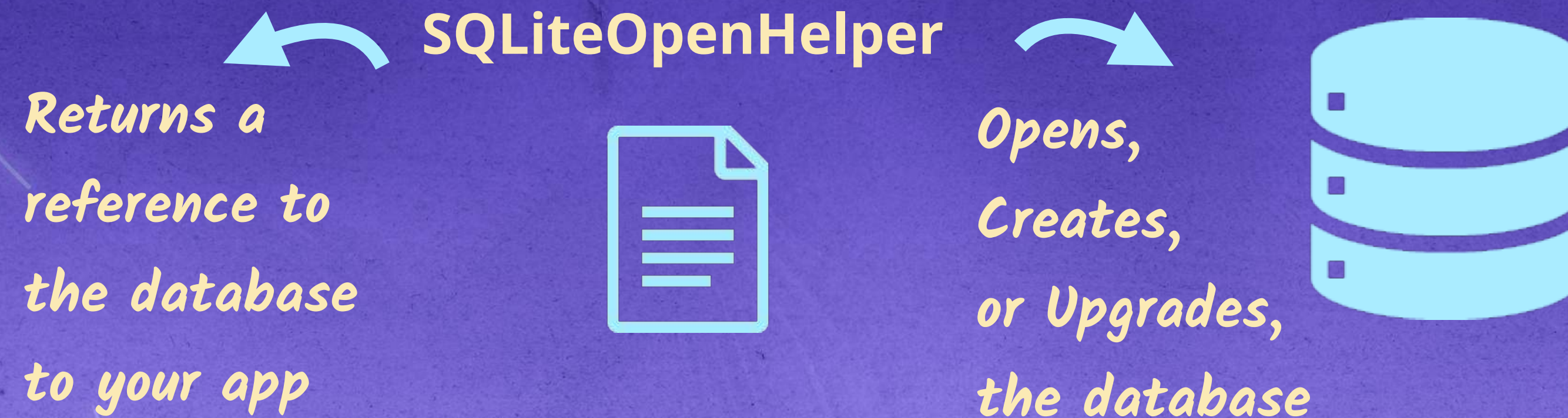
CandyContract.java

```
public class CandyContract {  
    public static final String DB_NAME = "candycoded.db";  
    public static final int DB_VERSION = 1;  
  
    private static final String SQL_CREATE_ENTRIES = " ... ";  
  
    private static final String SQL_DELETE_ENTRIES =  
        "DROP TABLE IF EXISTS " + CandyEntry.TABLE_NAME;  
  
    public static class CandyEntry implements BaseColumns {  
        ...  
    }  
}
```

Now that we have our Schema, or Contract, set up we want to create our database...

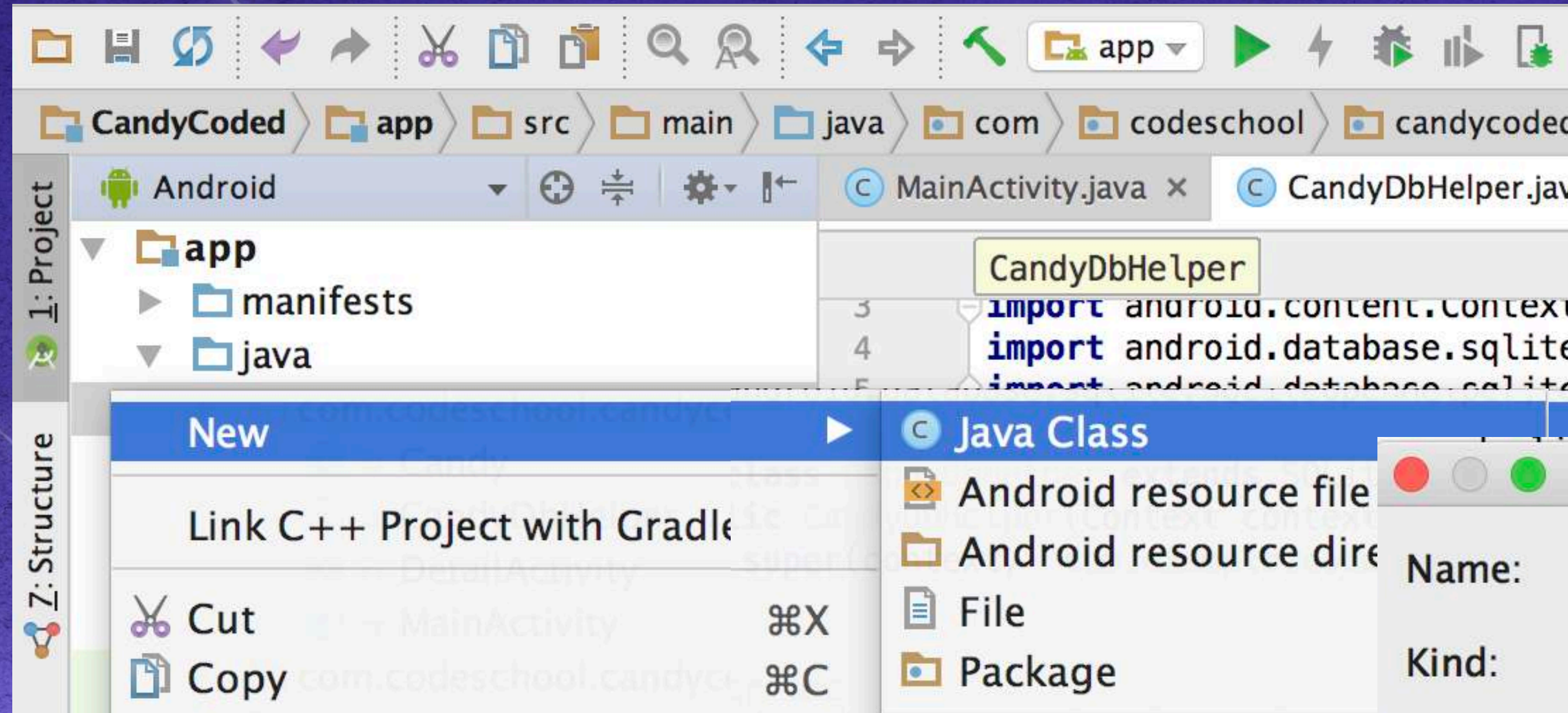
Using SQLiteOpenHelper to Manage Database Creation

The SQLiteOpenHelper class takes care of opening the database if it exists, creating it if it does not, and upgrading it as necessary.

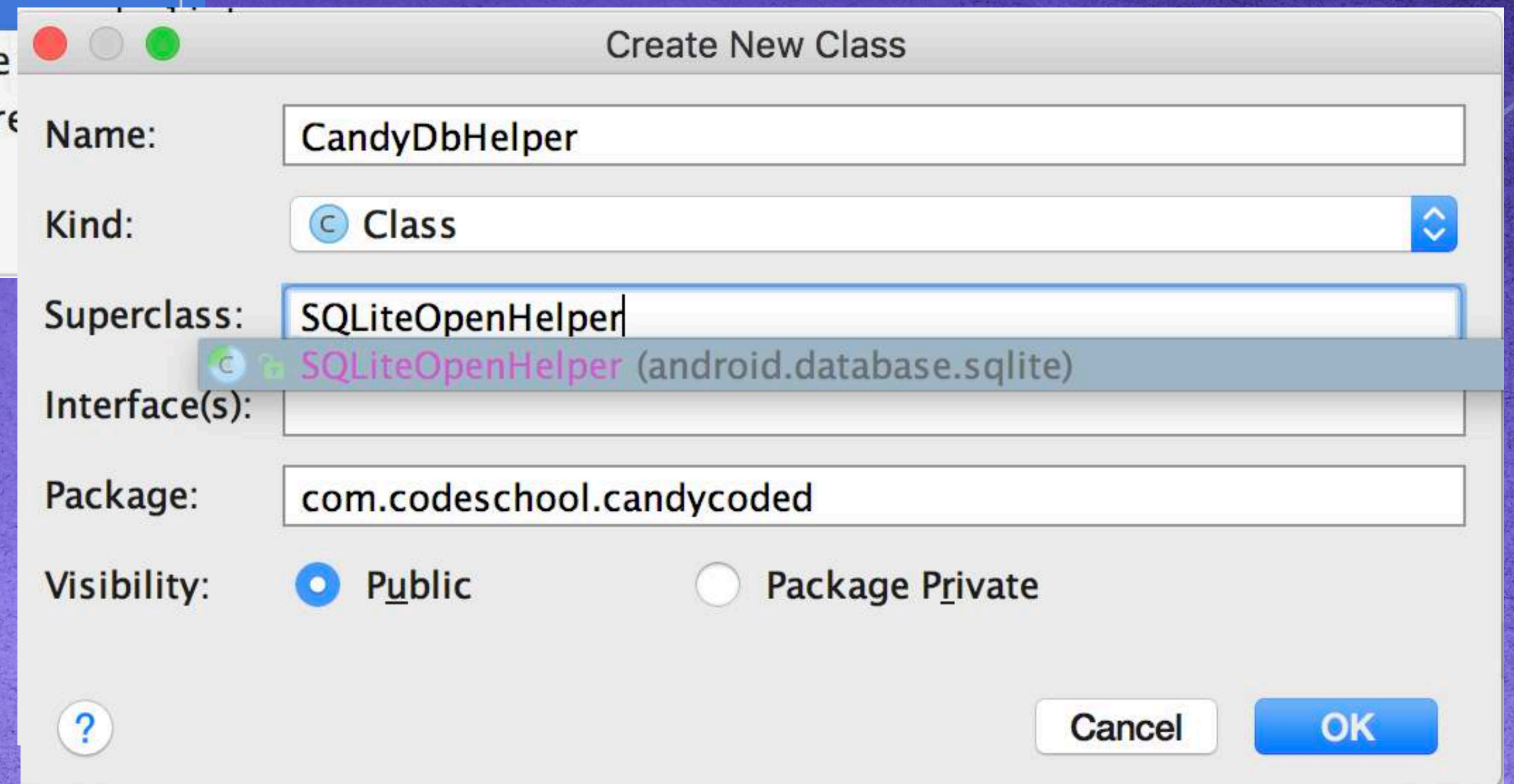


Creating Our Own SQLiteOpenHelper Subclass

To use the SQLiteOpenHelper class we need to create a new Java class called CandyDBHelper.



The class CandyDBHelper will extend the SQLiteOpenHelper class.



The Necessary Methods for the CandyDBHelper

The CandyDBHelper needs a constructor and needs to override the onCreate() and onUpgrade() methods.

CandyDbHelper.java

```
package com.codeschool.candycoded;

import android.database.sqlite.SQLiteOpenHelper;

public class CandyDbHelper extends SQLiteOpenHelper {

    public CandyDbHelper(Context context) {...}

    @Override
    public void onCreate(SQLiteDatabase db) {...}

    @Override
    public void onUpgrade(SQLiteDatabase db,
        int oldVersion, int newVersion) {...}

}
```

In this class we need to:

(1) Make a constructor that calls the super() method

(2) Override the onCreate() method

(3) Override the onUpgrade() method

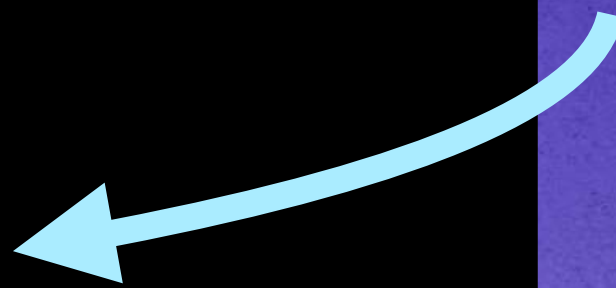
Adding a Constructor

Our constructor will call the `super()` method which calls the `SQLiteOpenHelper`'s constructor.

CandyDbHelper.java

```
...  
  
public class CandyDbHelper extends SQLiteOpenHelper {  
    public CandyDbHelper(Context context) {  
        super(context,  
            CandyContract.DB_NAME ,  
            null,  
            CandyContract.DB_VERSION);  
    }  
  
    @Override  
    public void onCreate(SQLiteDatabase db) {...}  
  
    @Override  
    public void onUpgrade(SQLiteDatabase db,  
        int oldVersion, int newVersion) {...}  
}
```

We pass in the context, the database name, and database version



(null is for the optional CursorFactory which we won't be using)

Adding the onCreate() Method

CandyDbHelper.java

```
...  
public class CandyDbHelper extends SQLiteOpenHelper {  
    public CandyDbHelper(Context context) {  
        super(...);  
  
        @Override  
        public void onCreate(SQLiteDatabase db) {  
            db.execSQL(CandyContract.SQL_CREATE_ENTRIES);  
        }  
  
        @Override  
        public void onUpgrade(SQLiteDatabase db,  
            int oldVersion, int newVersion) {...}  
    }  
}
```

The onCreate() method executes the statement we created earlier to create our table with the columns we have defined

Adding the onUpgrade() Method

CandyDbHelper.java

```
public class CandyDbHelper extends SQLiteOpenHelper {  
    public CandyDbHelper(Context context) {  
        super(...);  
    }  
  
    @Override  
    public void onCreate(SQLiteDatabase db) {  
        db.execSQL(CandyContract.SQL_CREATE_ENTRIES);  
    }  
  
    @Override  
    public void onUpgrade(SQLiteDatabase db,  
        int oldVersion, int newVersion) {...}  
        db.execSQL(CandyContract.SQL_DELETE_ENTRIES);  
        onCreate(db);  
    }  
}
```

To upgrade our database we will simply execute our statement to delete the table and then re-create our table

Now we have everything in place to begin using our database to store data!

Level 4 – Section 2

Storing Data

Adding Values to a SQLite Database

SUPER
SWEET
ANDROID
TIME



Saving Candies to the Database

We want to save Candies to the database after we get the Candy Array from the API. We can do this in the `onSuccess()` callback method as part of the `AsyncHttpClient` request.

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    ...
    @Override protected void onCreate(Bundle savedInstanceState) {
        ...
        AsyncHttpClient client = new AsyncHttpClient();
        client.get(...
            new TextHttpResponseHandler() {
                @Override public void onSuccess(...) {
                    Gson gson = new GsonBuilder().create();
                    Candy[] candies = gson.fromJson(response, Candy[].class);
                    adapter.clear();
                    for(Candy candy : candies) {
                        adapter.add(candy.name);
                    }
                }
            }
        );
        ...
    }
}
```

Remember the code we wrote for our network request?

This is where we'll add candies to our database

Saving Candies to the Database

We will create a method `addCandiesToDatabase()` to save each Candy to the database.

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    ...  
    @Override protected void onCreate(Bundle savedInstanceState) {  
        ...  
        AsyncHttpClient client = new AsyncHttpClient();  
        client.get(...  
            new TextHttpResponseHandler() {  
                @Override public void onSuccess(...) {  
                    Gson gson = new GsonBuilder().create();  
                    Candy[] candies = gson.fromJson(response, Candy[].class);  
                    adapter.clear();  
                    for(Candy candy : candies) {  
                        adapter.add(candy.name);  
                    }  
                    addCandiesToDatabase(candies);  
                }  
            }  
        );  
    }  
}
```

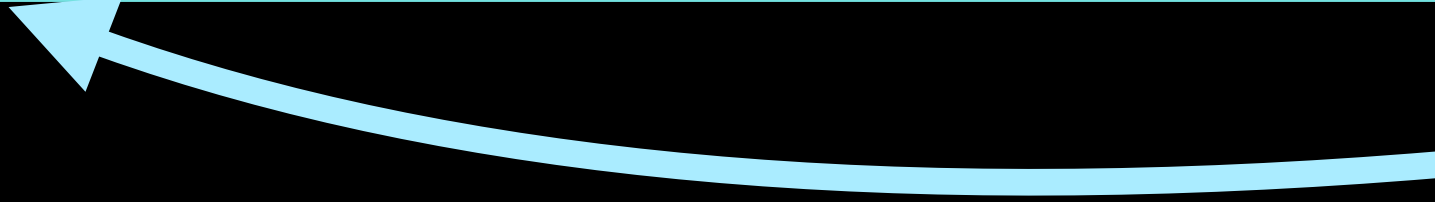
We'll call this method like this. But now we have to create the method!

Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private Candy[] candies;  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        ...  
    }  
  
}
```

We'll also add a CandyDbHelper variable that we can access throughout the MainActivity class



Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private Candy[] candies;  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        ...  
    }  
  
    public void addCandiesToDatabase(Candy[] candies) {  
    }  
}
```

The return type is void since we don't need to return anything we're just adding to the database

We need to pass in our array of Candy objects so that we can add each one to the database

Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
    ...  
    public void addCandiesToDatabase(Candy[] candies){  
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();
```

We'll store a reference to our database in a SQLiteDatabase variable

To access our database, we can get a writeable database from our candyDbHelper

```
}
```

```
...
```


Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
    ...  
    public void addCandiesToDatabase(Candy[] candies){  
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();
```

```
        for(Candy candy : candies) {
```



*Loop that reads each candy in our
array of candies*

```
        }
```

```
    }
```

```
    ...
```


Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
    ...  
    public void addCandiesToDatabase(Candy[] candies){  
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();  
  
        for(Candy candy : candies) {  
            ContentValues values = new ContentValues();  
        }  
    }  
    ...  
}
```

To insert values into the database we can use a ContentValues object

The ContentValues class does some data validation and ensures data gets inserted in the correct format

Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
    ...  
    public void addCandiesToDatabase(Candy[] candies){  
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();  
  
        for(Candy candy : candies) {  
            ContentValues values = new ContentValues();  
            values.put(CandyEntry.COLUMN_NAME_NAME, candy.name);  
        }  
    }  
    ...  
}
```



We then put each column name and value into the ContentValues object with the put() method

Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);
    ...
    public void addCandiesToDatabase(Candy[] candies){
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();

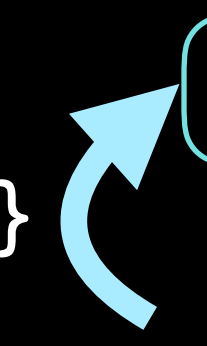
        for(Candy candy : candies) {
            ContentValues values = new ContentValues();
            values.put(CandyEntry.COLUMN_NAME_NAME, candy.name);
            values.put(CandyEntry.COLUMN_NAME_PRICE, candy.price);
            values.put(CandyEntry.COLUMN_NAME_DESC, candy.description);
            values.put(CandyEntry.COLUMN_NAME_IMAGE, candy.image);
        }
    }
    ...
}
```

And we'll do this for the rest of our candy's properties

Saving Candies to the Database

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
    ...  
    public void addCandiesToDatabase(Candy[] candies){  
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();  
  
        for(Candy candy : candies) {  
            ContentValues values = new ContentValues();  
            values.put(CandyEntry.COLUMN_NAME_NAME, candy.name);  
            values.put(CandyEntry.COLUMN_NAME_PRICE, candy.price);  
            values.put(CandyEntry.COLUMN_NAME_DESC, candy.description);  
            values.put(CandyEntry.COLUMN_NAME_IMAGE, candy.image);  
  
            db.insert(CandyEntry.TABLE_NAME, null, values);  
        }  
    }  
    ...  
}
```



Finally we can use the insert method to insert the row into our database

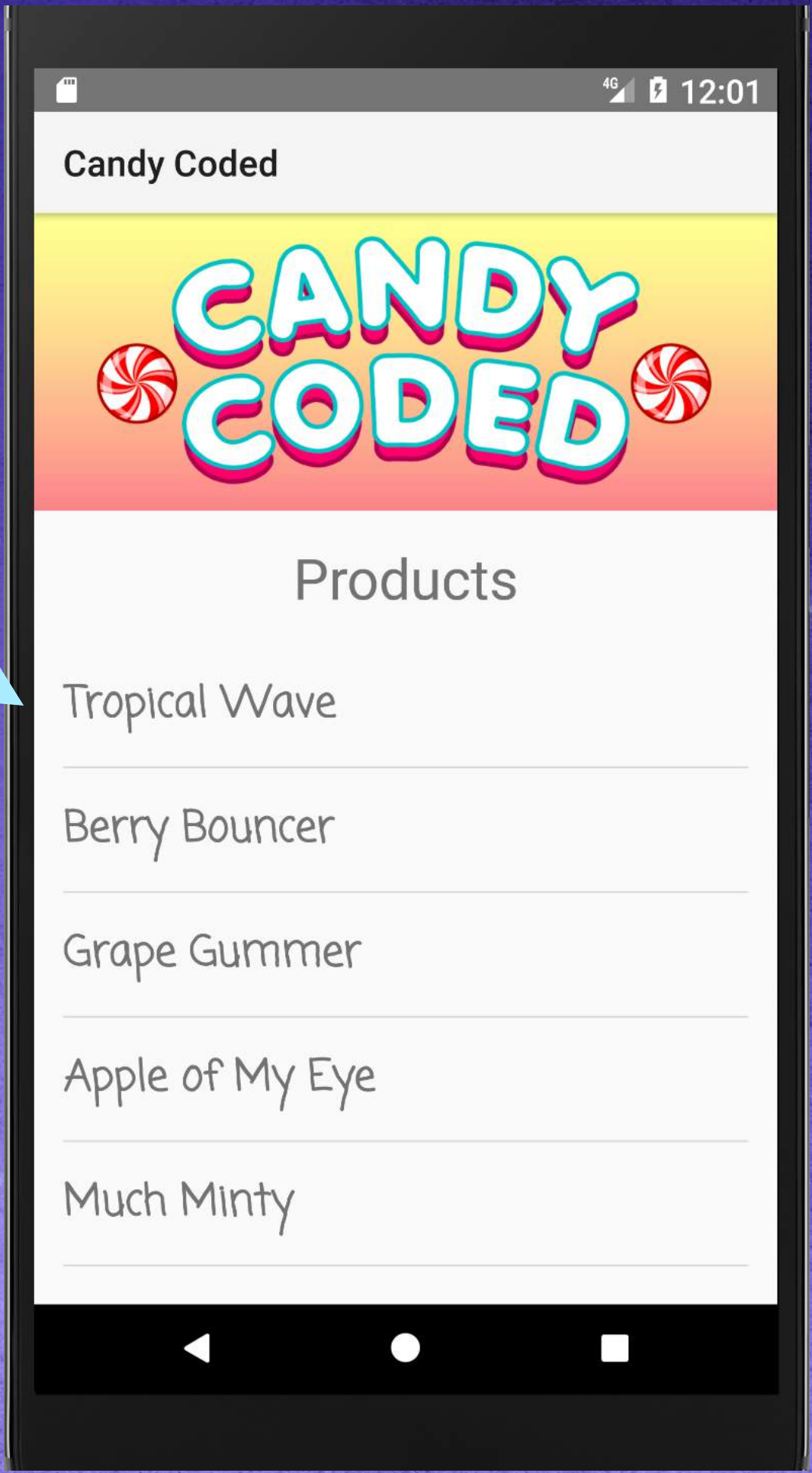
Now We Have Data in Our Database

Now that we have our candies in our database, the next step is to use them in our app.

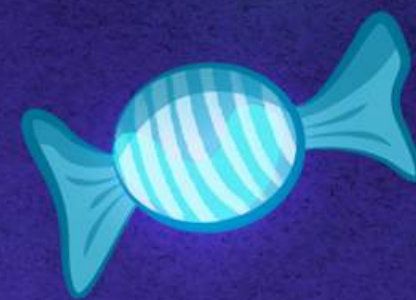
 Candy Database
Candy Table

_ID	Name	Price	Description	Image
1	Much Minty	4.50	This peppermint ...	...
2	So Fresh	5.50	The wintergreen ...	...
3	Uni-Pop	9.99	The sugary magic...	...
...	...	...	...	...

Next we need to query our database for candies and add them to our app



SUPER
SWEET
ANDROID
TIME



Level 5 – Section 1

Querying a SQLite Database

Cursors & Cursor Adapters

SUPER
SWEET
ANDROID
TIME



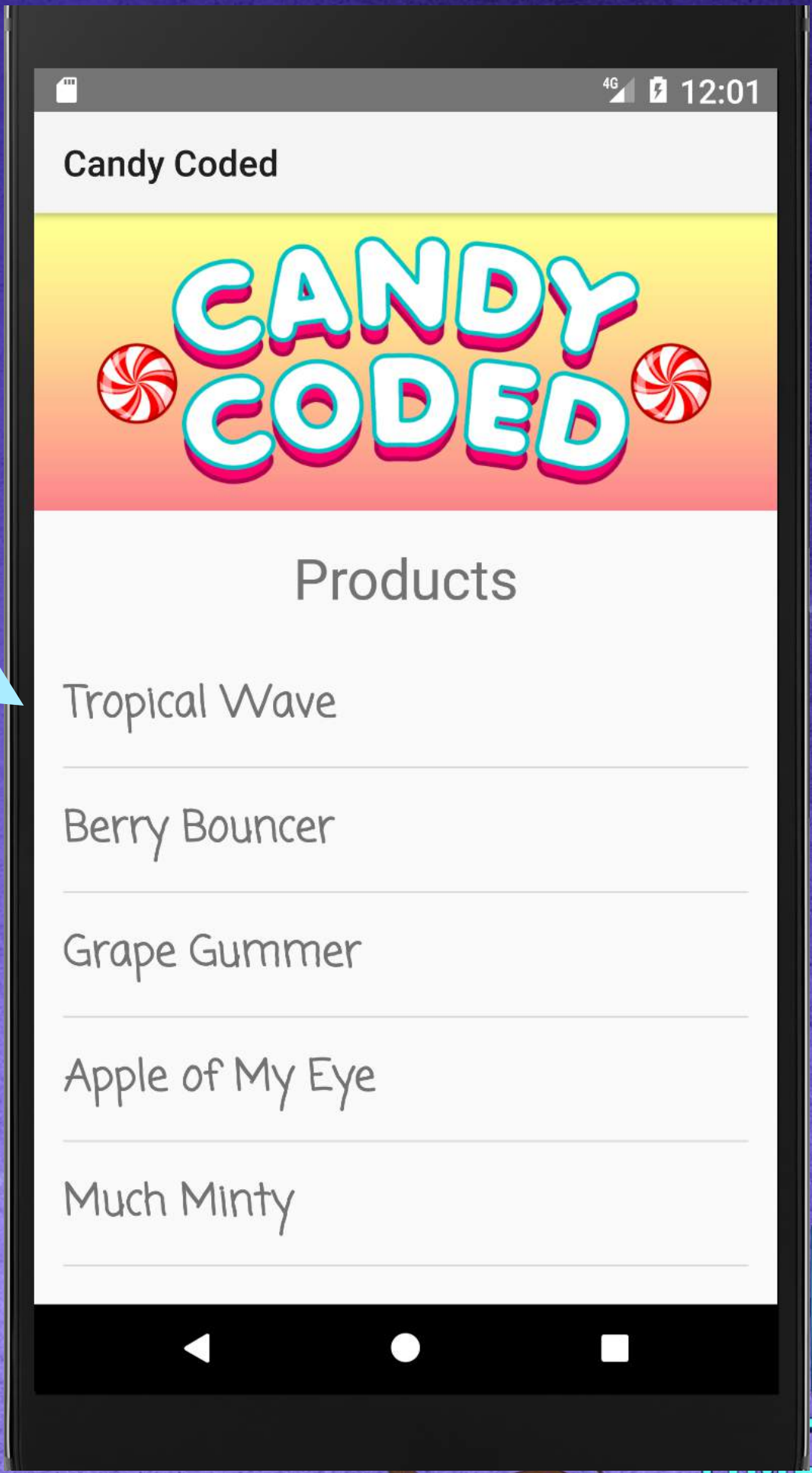
Now We Have Data in Our Database

Now that we have our candies in our database, the next step is to use them in our app.



Name	Price	Description	Image
Much Minty	4.50	This peppermint ...	...
So Fresh	5.50	The wintergreen ...	...
Uni-Pop	9.99	The sugary magic...	...
...			

Next we need to query our database for candies and add them to our app



Using Raw SQL to Get All of the Candies

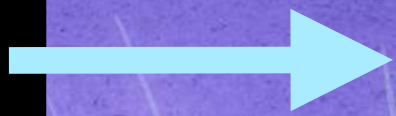
We can get values from the table with Select.

Name	Price	Description	Image
So Fresh	5.50	The wintergreen ...	...
Uni-Pop	9.99	The sugary magic...	...

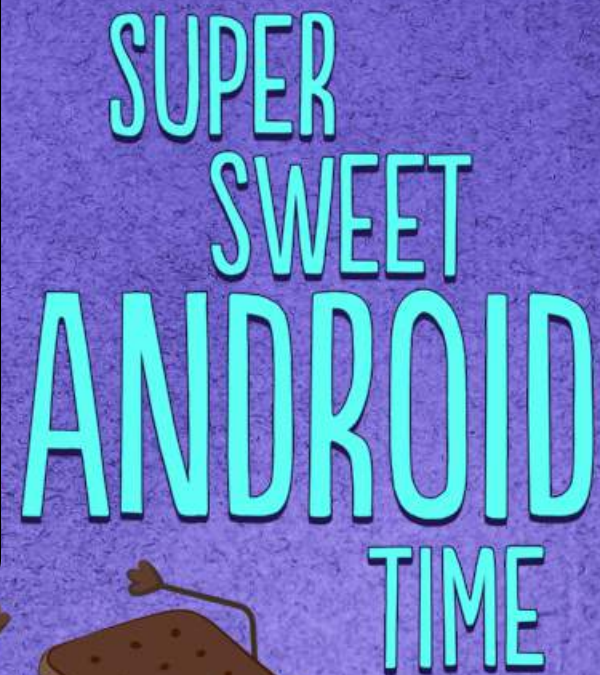
```
SELECT * FROM candy
```



*SELECT * returns all of the columns and will return all of the candy rows which is what we want*



```
+-----+-----+-----+-----+
| Name   | Price | Desc  | Image |
+-----+-----+-----+-----+
| So Fresh | 4.50  | ...   | ...   |
| Uni-Pop | 5.50  | ...   | ...   |
```



Querying Our SQLite Database

We can query a SQLiteDatabase object with the `rawQuery()` method like below:

```
Cursor cursor = db.rawQuery("SELECT * FROM candy", null);
```

A Cursor exposes the results from a query

db is our SQLiteDatabase

We pass our SQL query as a String to the `rawQuery()` method

Name	Price	Description	Image
Much Minty	4.50	This peppermint ...	...
So Fresh	5.50	The wintergreen ...	...
Uni-Pop	9.99	The sugary magic...	...

Our query will return all of the candy rows and we can navigate the resulting rows with our Cursor

SUPER
SWEET
ANDROID
TIME



How a Cursor Works

Cursors store query result records in rows and have methods to access and iterate through the records.

```
Cursor cursor = db.rawQuery("SELECT * FROM candy", null);
```

*Cursor's initial
position (i.e. -1)*



Name	Price	Description	Image
Much Minty	4.50	This peppermint ...	...
So Fresh	5.50	The wintergreen ...	...
Uni-Pop	9.99	The sugary magic...	...



How a Cursor Works

Cursors store query result records in rows and have methods to access and iterate through the records.

```
Cursor cursor = db.rawQuery("SELECT * FROM candy", null);
```

After calling
`cursor.moveToNext()`



Name	Price	Description	Image
Much Minty	4.50	This peppermint ...	...
So Fresh	5.50	The wintergreen ...	...
Uni-Pop	9.99	The sugary magic...	...

```
int index = cursor.getColumnIndexOrThrow("name");  
String candyName = cursor.getString(index);
```

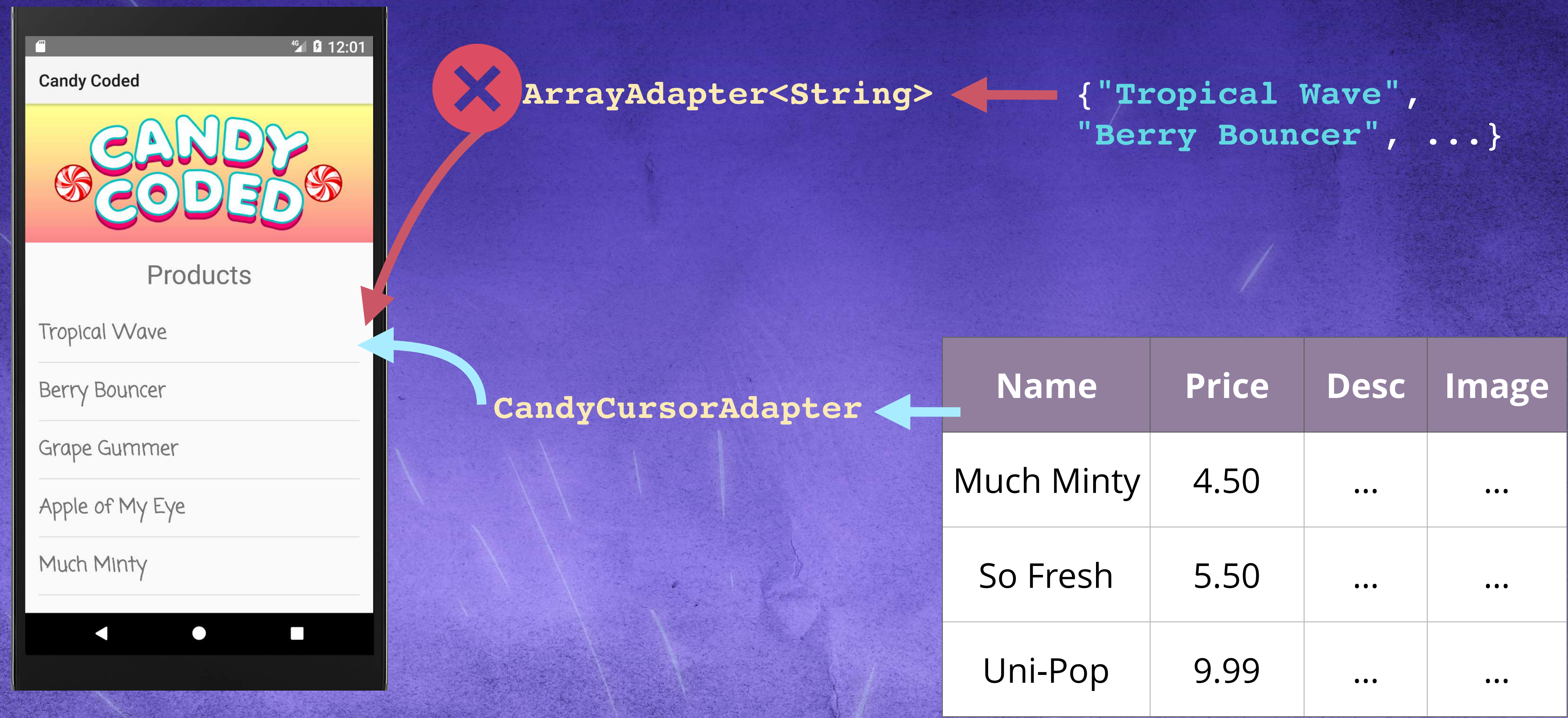
candyName is now "Much Minty"

SUPER
SWEET
ANDROID
TIME



How Do We Get Our Candy Names in Our

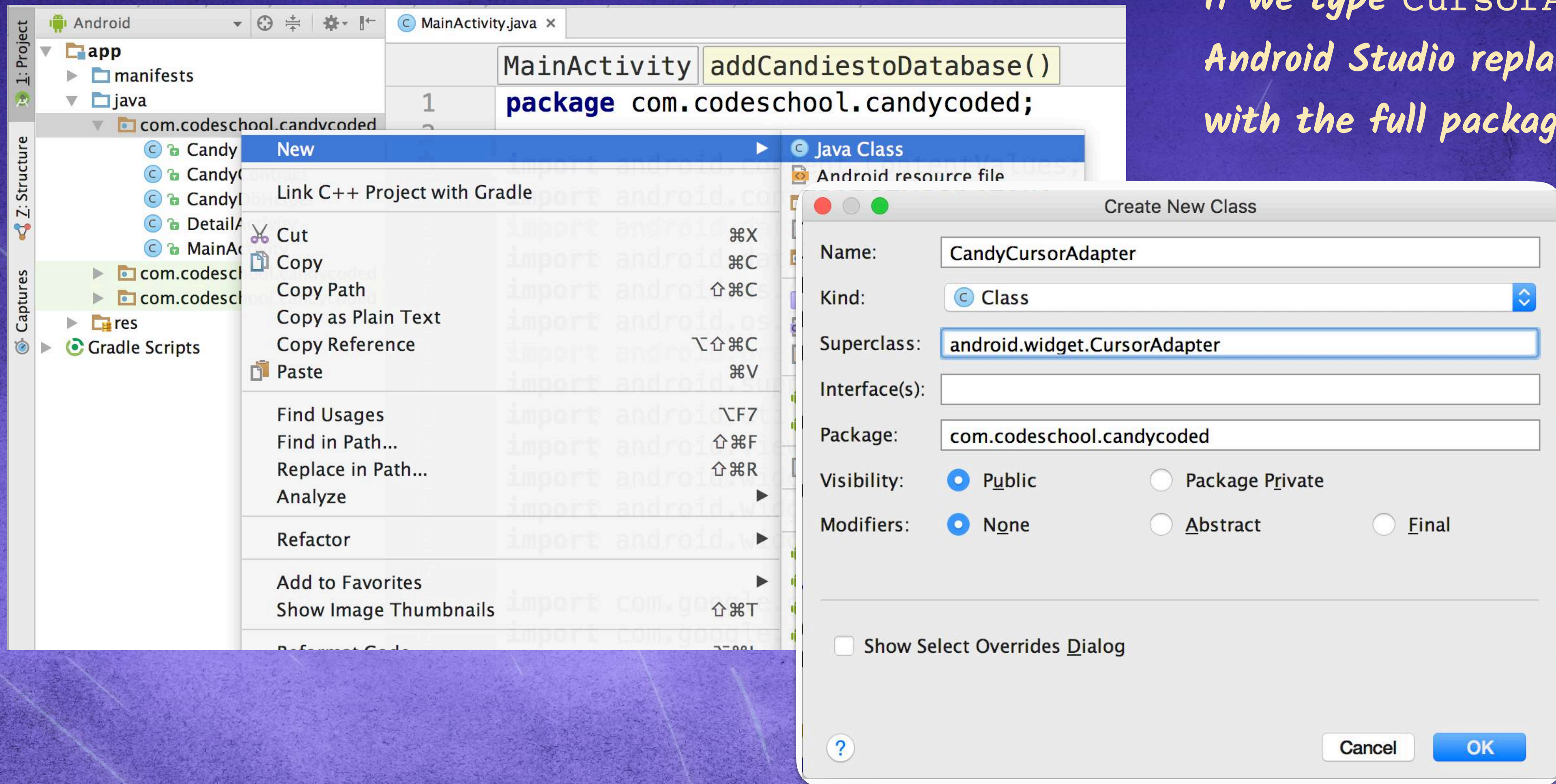
Instead of using an ArrayAdapter to populate our ListView, we can use a CursorAdapter that gets data directly from our Database.



Creating a CursorAdapter Subclass

To use a CursorAdapter to populate our ListView we need to create a CursorAdapter Subclass, which we will name CandyCursorAdapter.

If we type CursorAdapter, Android Studio replaces it with the full package name



SUPER
SWEET
ANDROID
TIME



The CandyCursorAdapter Class

Since CandyCursorAdapter **extends** CursorAdapter it has to have certain methods listed below.

CandyCursorAdapter.java

```
package com.codeschool.candycoded;  
import ...
```

```
public class CandyCursorAdapter extends CursorAdapter {
```

```
    public CandyCursorAdapter(Context context, Cursor c) {...}
```

```
    @Override
```

```
    public View newView(Context context, Cursor cursor,  
                        ViewGroup parent) {...}
```

```
    @Override
```

```
    public void bindView(View view, Context context,  
                        Cursor cursor) {...}
```

```
}
```

In this class we need to:

(1) Make a constructor

(2) Override the newView() method

(3) Override the bindView() method

The CandyCursorAdapter Class

CandyCursorAdapter.java

```
...  
public class CandyCursorAdapter extends CursorAdapter {  
  
    public CandyCursorAdapter(Context context, Cursor c) {  
        super(context, c, false);  
    }  
  
    @Override  
    public View newView(Context context, Cursor cursor,  
                        ViewGroup parent) {...}  
  
    @Override  
    public void bindView(View view, Context context,  
                        Cursor cursor) {...}  
}
```

The constructor will call the super() method that needs the context, the Cursor, and false for re-query

The CandyCursorAdapter Class

CandyCursorAdapter.java

```
...
public class CandyCursorAdapter extends CursorAdapter {

    public CandyCursorAdapter(Context context, Cursor c) {
        super(context, c, false);
    }

    @Override
    public View newView(Context context,
                        Cursor cursor, ViewGroup parent) {
        return LayoutInflater.from(context).inflate(
            R.layout.list_item_candy, parent, false);
    }

    @Override public void bindView(View view, Context context,
                                    Cursor cursor) {...}
}
```

This method inflates the view and returns it so it can be displayed

We need to tell it to what layout to use: list_item_candy layout we created in Try Android

The CandyCursorAdapter Class

CandyCursorAdapter.java

```
...
public class CandyCursorAdapter extends CursorAdapter {
    public CandyCursorAdapter(Context context, Cursor c) {...}

    @Override public View newView(...) {...}

    @Override public void bindView(View view, Context context,
                                    Cursor cursor) {
        TextView textView = (TextView) view.findViewById(
            Cursor cursor, R.id.text_view_candy);
        String candyName = cursor.getString(
            cursor.getColumnIndexOrThrow("name"));
        textView.setText(candyName);
    }
}
```

In this method we set the elements of our view

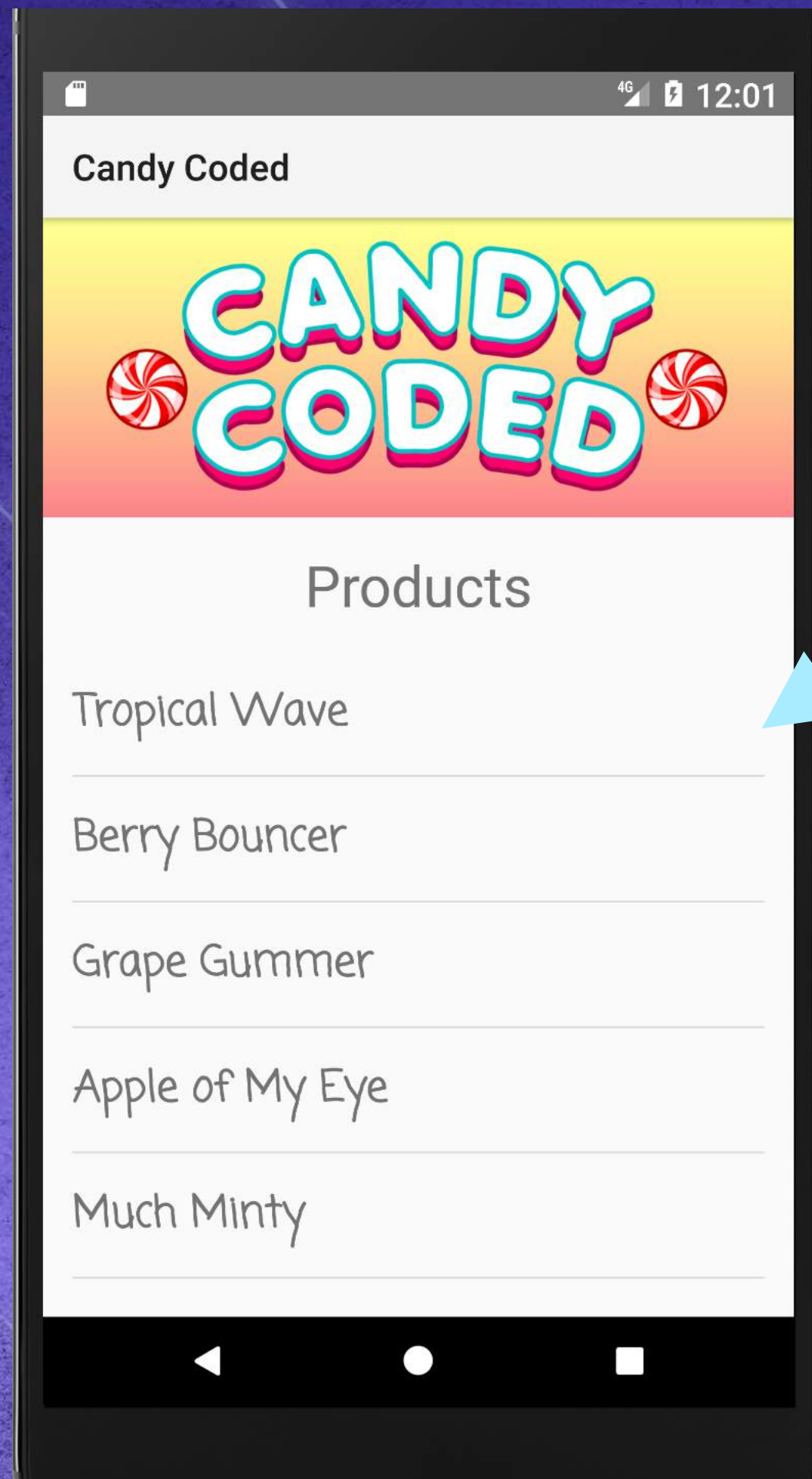
(1) We need to get TextView to fill

(2) Then we get the column in our database to fill it with

(3) Then we use the setText() method to connect the two

Using the CandyCursorAdapter to Put Candy in Our ListView

Now we can use our CursorAdapter to get data directly from our Database into our ListView.



CandyCursorAdapter

Name	Price	Desc	Image
Much Minty	4.50	...	...
So Fresh	5.50	...	...
Uni-Pop	9.99	...	...

To use our new CandyCursorAdapter we need to refactor the code in MainActivity.java a bit

SUPER
SWEET
ANDROID
TIME



Refactoring Our Code to Use the CandyCursorAdapter

MainActivity.java

```
...
public class MainActivity extends AppCompatActivity {
    ...

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        ...
        final ArrayList<String> candyList = new ArrayList<String>();
        candy_list.add("Tropical Wave");
        ...
        final ArrayAdapter<String> adapter = new ArrayAdapter<String>(...);
        ...
    }
}
```



We can remove the code that creates the ArrayList and the ArrayAdapter since we'll be using our Database and CandyCursorAdapter instead.

Refactoring Our Code to Use the CandyCursorAdapter

MainActivity.java

```
...  
public class MainActivity extends AppCompatActivity {  
    private CandyCursorAdapter adapter;  
    private CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {
```

```
        ...  
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();  
        Cursor cursor = db.rawQuery("SELECT * FROM candy", null);  
  
        adapter = new CandyCursorAdapter(this, cursor);
```

```
        ...  
        listView.setAdapter(adapter);
```

```
        ... We will first query for all candies
```

```
    } Then we can create a CandyCursorAdapter from the results in the cursor  
}
```


Refactoring Our Code to Use the CandyCursorAdapter

MainActivity.java

...

```
AsyncHttpClient client = new AsyncHttpClient();
```

```
client.get(
```

```
    "https://....herokuapp.com/main/api",
```

```
    new TextHttpResponseHandler() {
```

```
        ...
```

```
        @Override public void onSuccess(int status, Header[] headers,
                                         String response) {
```

```
            Gson gson = new GsonBuilder().create();
```

```
            Candy[] candies = gson.fromJson(response, Candy[].class);
```

```
            adapter.clear();
```

```
            for(Candy candy : candies) {
                adapter.add(candy.name);
            }
```

```
            addCandiestoDatabase(candies);
```

```
        }
```

```
    });
```

...

*We can also
remove the code
to update the
ArrayAdapter
after our request*



Refactoring Our Code to Use the CandyCursorAdapter

MainActivity.java

...

```
AsyncHttpClient client = new AsyncHttpClient();
```

```
client.get(
```

```
    "https://....herokuapp.com/main/api",
```

```
    new TextHttpResponseHandler() {
```

```
        ...
```

```
        @Override public void onSuccess(int status, Header[] headers,
                                         String response) {
```

```
            Gson gson = new GsonBuilder().create();
```

```
            Candy[] candies = gson.fromJson(response, Candy[].class);
```

```
            addCandiesToDatabase(candies);
```

*Updating our
CursorAdapter
with the latest
Database entries*

```
            SQLiteDatabase db = candyDbHelper.getWritableDatabase();
            Cursor cursor = db.rawQuery("SELECT * FROM candy", null);
            adapter.changeCursor(cursor);
```

```
        }
    });
```

...

Refactoring How We Pass Data to the DetailActivity

MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    ...
    @Override protected void onCreate(Bundle savedInstanceState) {
        ...
        SQLiteDatabase db = candyDbHelper.getWritableDatabase();
        Cursor cursor = db.rawQuery("SELECT * FROM candy", null);
        adapter = new CandyCursorAdapter(this, cursor);

        listView.setOnItemClickListener(
            new AdapterView.OnItemClickListener() {
                @Override public void onItemClick(
                    AdapterView<?> adapterView, View view, int i, long l) {
                    Intent detailIntent = new Intent(this, DetailActivity.class);
                    detailIntent.putExtra("position", i);
                    startActivity(detailIntent);
                }
            });
        ...
    }
}
```

Instead of passing each candy property to the Detail Activity, we can pass just i and then query the database in the Detail Activity

Level 5 – Section 2

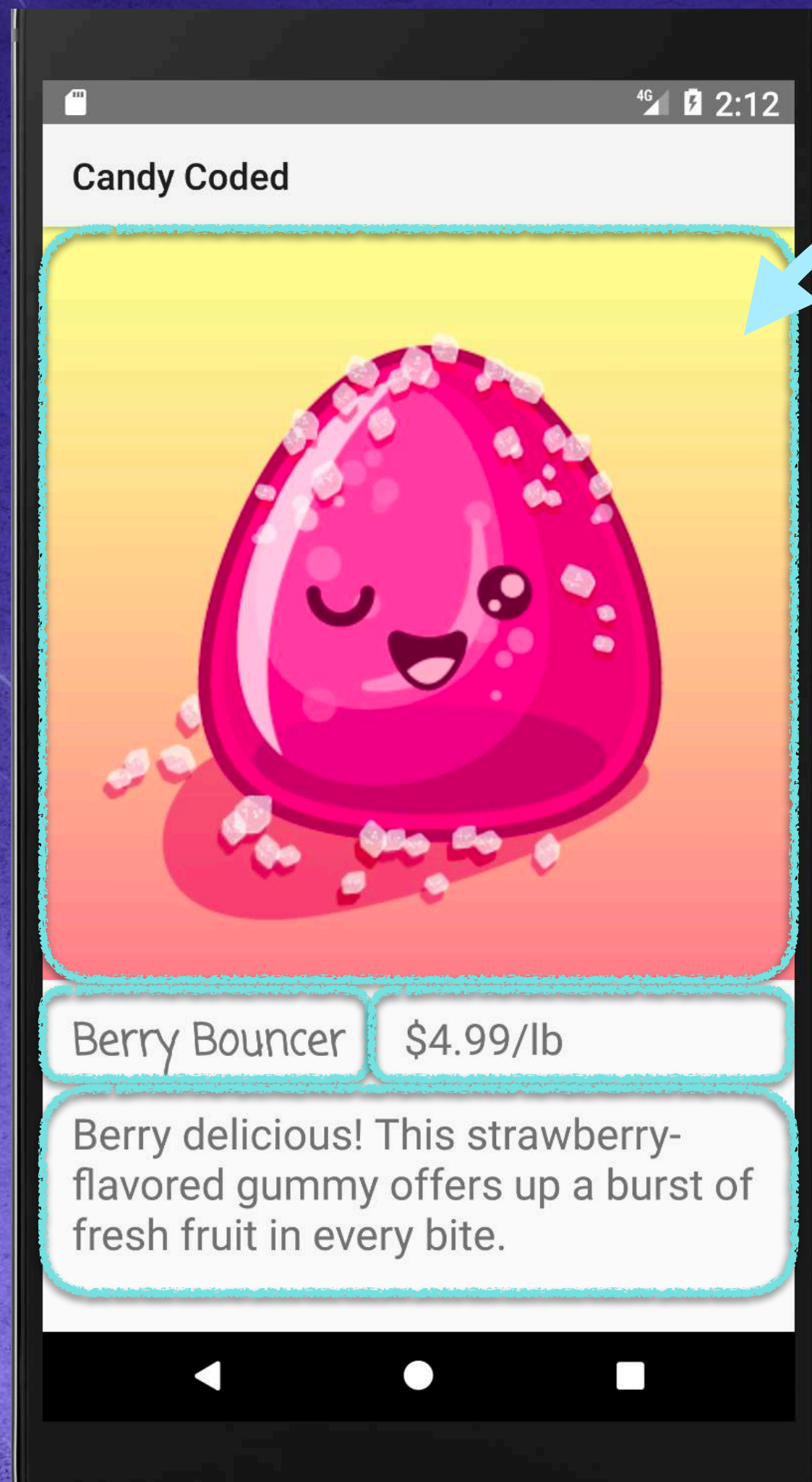
Querying a SQLite Database

Getting Data in the Detail Activity

SUPER
SWEET
ANDROID
TIME



Refactoring the Detail Activity to Query Our Database



Instead of passing each of these values (the candy name, price, image, and description) individually to the DetailActivity, we are passing the id to use in a database query



Refactoring the DetailActivity to Query the Database

DetailActivity.java

```
...
public class DetailActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_detail);
        Intent intent = DetailActivity.this getIntent();

        ❌ String candyName = "";
        if (intent != null && intent.hasExtra("candy_name")) {
            candyName = intent.getStringExtra("candy_name");
        }

        TextView textViewName = (TextView) this.findViewById(
            R.id.text_view_name);
        textViewName.setText(candyName);
        ...
    }
}
```

Before we were passing in each value to display, but now we need to query the database instead

So we can remove this code that looked up the extra properties for the name, description, price and image

Refactoring the DetailActivity to Query the Database

DetailActivity.java

```
...  
public class DetailActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_detail);  
        Intent intent = DetailActivity.this.getIntent();
```

```
        if (intent.hasExtra("position")) {
```

*First we'll make sure our intent has the
extra "position" value*

```
        }
```


Refactoring the DetailActivity to Query the Database

DetailActivity.java

```
...
public class DetailActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_detail);
        Intent intent = DetailActivity.this getIntent();

        if (intent.hasExtra("position")) {
            int position = intent.getIntExtra("position", 0);

            We can then get the position value, which is  
row that was selected in the ListView
        }
    }
}
```


Refactoring the DetailActivity to Query the Database

DetailActivity.java

...

```
if (intent.hasExtra("position")) {  
    int position = intent.getIntExtra("position", 0);  
    CandyDbHelper candyDbHelper = new CandyDbHelper(this);  
    SQLiteDatabase db = candyDbHelper.getWritableDatabase();  
    Cursor cursor = db.rawQuery("SELECT * FROM candy", null);  
    cursor.moveToPosition(position);  
}
```

We can query all candies and then move our cursor to the selected position

}

...

Refactoring the DetailActivity to Query the Database

DetailActivity.java

```
...  
    if (intent.hasExtra("position")) {  
        int position = intent.getIntExtra("position", 0);  
        ...  
        Cursor cursor = db.rawQuery("SELECT * FROM candy", null);  
        cursor.moveToPosition(position);
```

```
        int columnIndex = cursor.getColumnIndexOrThrow(  
            CandyEntry.COLUMN_NAME_NAME);
```

We want the value in the name column so we can display the candy's name

```
    }  
    ...  
    We can use the getColumnIndexOrThrow() to get the index of our name column. The "orThrow" means it will throw an error if it's not there
```


Refactoring the DetailActivity to Query the Database

DetailActivity.java

```
...  
    if (intent.hasExtra("position")) {  
        int position = intent.getIntExtra("position", 0);  
        ...  
        Cursor cursor = db.rawQuery("SELECT * FROM candy", null);  
        cursor.moveToPosition(position);  
  
        int columnIndex = cursor.getColumnIndexOrThrow(  
            CandyEntry.COLUMN_NAME_NAME);  
        String candyName = cursor.getString(columnIndex);  
  
        Then we can get the candy's name with the  
        getString() method and pass the columnIndex  
    }  
    ...
```


Refactoring the DetailActivity to Query the Database

DetailActivity.java

```
...  
    if (intent.hasExtra("position")) {  
        int position = intent.getIntExtra("position", 0);  
        ...  
        Cursor cursor = db.rawQuery("SELECT * FROM candy", null);  
        cursor.moveToPosition(position);  
  
        String candyName = cursor.getString(cursor.getColumnIndexOrThrow(  
            CandyEntry.COLUMN_NAME_NAME));  
        String candyPrice = cursor.getString(cursor.getColumnIndexOrThrow(  
            CandyEntry.COLUMN_NAME_PRICE));  
        String candyImage = cursor.getString(cursor.getColumnIndexOrThrow(  
            CandyEntry.COLUMN_NAME_IMAGE));  
        String candyDesc = cursor.getString(cursor.getColumnIndexOrThrow(  
            CandyEntry.COLUMN_NAME_DESC));  
    }  
    ...
```

*We want to do the same thing for the price, image,
and description values*

}

...

Refactoring the DetailActivity to Query the Database

DetailActivity.java

```
...
    if (intent.hasExtra("position")) {
        ...

        ((TextView) DetailActivity.this.findViewById(R.id.text_view_name))
            .setText(candyName);

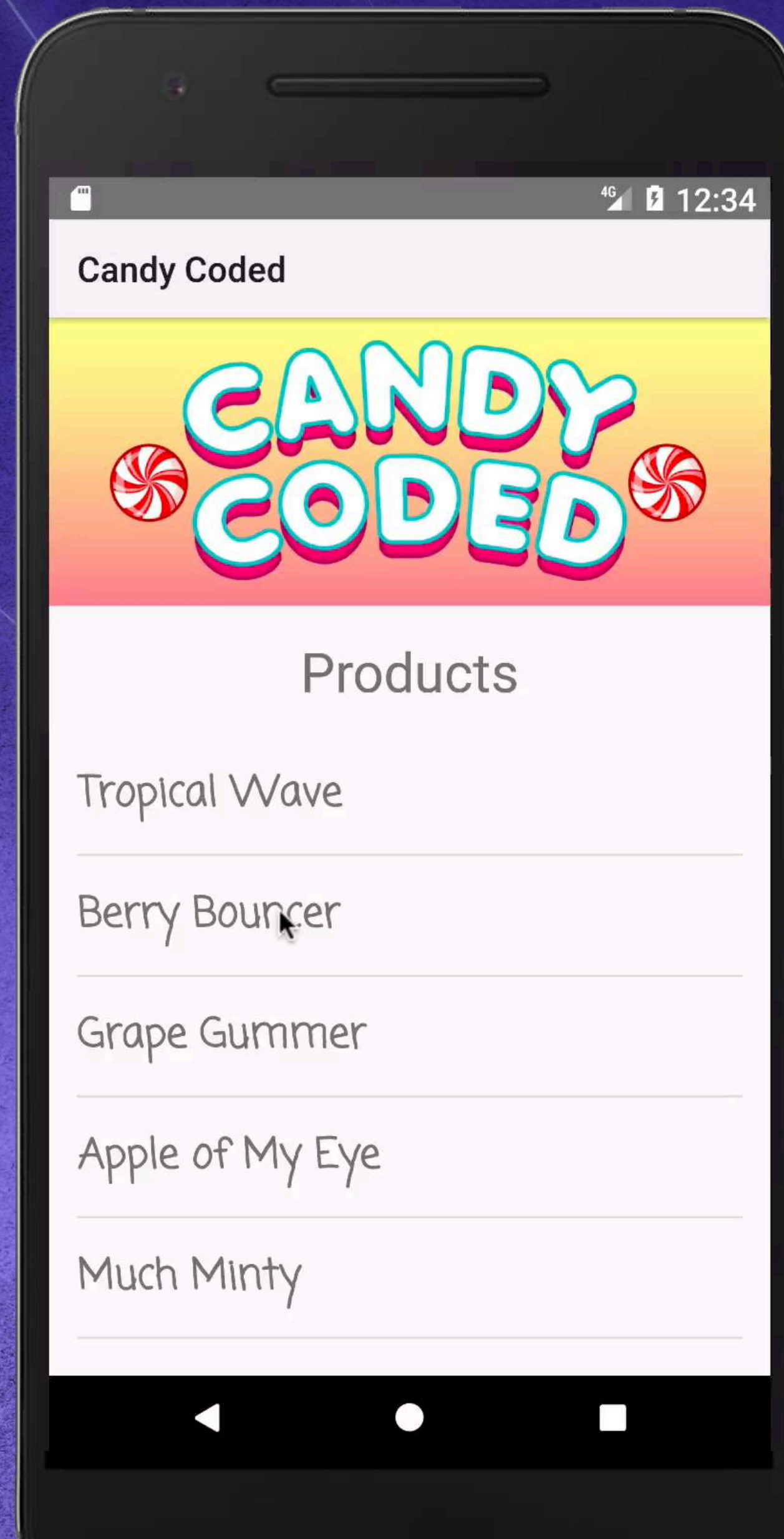
        ((TextView) DetailActivity.this.findViewById(R.id.text_view_price))
            .setText(candyPrice);

        ((TextView) DetailActivity.this.findViewById(R.id.text_view_desc))
            .setText(candyDesc);

        Picasso.with(DetailActivity.this).load(candyImage).into(
            (ImageView) DetailActivity.this.findViewById(R.id.image_view_candy));
    }
...
```

This code still works like before because our name, price, description, and image variable names haven't changed

The Detail Activity Working with Our Database Data



Now if we run our app we can see our Detail Activity working with the data queried from our database!

SUPER
SWEET
ANDROID
TIME



SUPER SWEET ANDROID TIME

